IntegraONE

Blue Team

Fundamentals

Table of Contents

Introduction

This student manual is not meant to be a textbook on Blue Teaming. It provides a brief introduction to what a Blue Team is and what it does, and then walks the student through thirteen exercises that a typical Blue Team member might do any given day. At the end is a cap stone exercise that ties everything together.

What is the Blue Team?

The **Blue Team** represents the defensive side of cybersecurity. In contrast to offensive security roles (often called the **Red Team**), which simulate attacks to identify vulnerabilities, the Blue Team focuses on protecting an organization's networks, systems, data, and assets from real-world threats. Their mission is to maintain a strong security posture, continuously monitor suspicious activity, detect intrusions early, analyze potential incidents, respond effectively, and improve defenses over time.

In military-inspired terminology commonly used in cybersecurity exercises:

- **Red Team** → Plays offense: ethical hackers and penetration testers who emulate adversaries to test defenses and uncover weaknesses.
- **Blue Team** → Plays defense: the protectors who monitor, detect, respond, and harden systems against those attacks.
- **Purple Team** → A collaborative approach: combines red and blue efforts to share knowledge in real time, validate detections, reduce false positives, and accelerate improvements through joint exercises and feedback loops.

The Blue Team operates under the principle of "**assume breach**" - acknowledging that perfect prevention is impossible, so the focus shifts to rapid detection, containment, and recovery while minimizing impact. This mindset drives proactive measures like threat hunting, detection engineering, and incident response, rather than purely reactive firefighting.

Core Responsibilities of the Blue Team

Blue Team professionals (often working within a **Security Operations Center**, or SOC) handle a wide range of defensive tasks, including:

- Continuous monitoring of logs, network traffic, endpoints, and security tools for indicators of compromise.

- Triage and investigation of alerts to separate true threats from noise (false positives).
- Detection rule creation and tuning (using formats like Sigma) to catch attacker behaviors.
- Incident response: containing breaches, eradicating threats, and recovering systems.
- Threat hunting: proactively searching for hidden adversaries using hypothesis-driven or intelligence-led approaches.
- Vulnerability management, patching, and hardening systems.
- Building and maintaining defensive playbooks, baselines of "normal" activity, and post-incident lessons learned.
- Collaborating with other teams (e.g., IT, threat intelligence, compliance) to align security with business needs.

Key Roles in Blue Team Operations

Many Blue Team members start in a **Security Operations Center (SOC)**, where analysts are typically tiered by experience and responsibility:

- **Tier 1 (Triage/SOC Analyst)**: Front-line monitoring, initial alert handling, basic investigations, and escalation of serious incidents.
- **Tier 2 (Advanced Analyst/Incident Responder)**: Deeper forensic analysis, malware triage, threat hunting, and leading response efforts.
- **Tier 3 (Senior/Threat Hunter/Engineer)**: Expert-level hunting, detection engineering, tool development, and strategic improvements.

Other common Blue Team roles include Detection Engineer, Threat Hunter, Incident Responder, SOC Manager, and specialists in areas like malware analysis or network defense.

The “Assume Breach” Mindset

The **Assume Breach** mindset is one of the most transformative shifts in modern cybersecurity thinking. It moves away from the traditional "castle-and-moat" or perimeter-focused model—where everything inside the network firewall is implicitly trusted once past the outer defenses—and instead starts with a realistic, often pessimistic, premise: **a breach has either already occurred or is inevitable given today's threat landscape**.

This philosophy underpins much of contemporary defensive strategy, including **Zero Trust** architectures (as explicitly outlined by Microsoft and NIST), threat hunting, detection engineering, and incident response readiness.

Core Idea: "Prevention Is Important, But Not Enough"

No defense is perfect. Sophisticated attackers (nation-states, ransomware groups, advanced persistent threats) use living-off-the-land techniques, supply-chain compromises, credential abuse, zero-days, and social engineering to bypass even strong perimeter controls. Traditional perimeter security assumes that if you keep bad actors out at the gate, you're safe inside. History (SolarWinds, MOVEit, Colonial Pipeline, countless ransomware campaigns) proves that's no longer sufficient.

Assume Breach flips the question:

- Instead of "How do we stop every attack?"
- It asks: "What do we do when attackers are already inside?"

By assuming compromise is already present (or imminent), defenders design systems, processes, and monitoring to:

- Minimize damage ("blast radius")
- Detect lateral movement and persistence quickly
- Respond and recover rapidly with minimal business impact

Key Principles and Implications:

1. **Shift Focus to Detection, Containment, and Response**
    - Prevention remains essential (patch, harden, MFA, etc.), but the bulk of effort goes into visibility, behavioral analytics, and rapid isolation.
    - Invest heavily in logging, EDR, SIEM, network monitoring, and threat hunting so you can find adversaries even when they're using legitimate tools (LOLBins).

2. **Minimize Blast Radius and Lateral Movement**
    - Network segmentation (micro-segmentation in Zero Trust)
    - Least-privilege access everywhere
    - Identity-based controls rather than network-location trust
    - Protect "crown jewels" (domain controllers, sensitive databases, backups) first

3. **Continuous Verification ("Never Trust, Always Verify")**
    - Every access request is scrutinized, regardless of source (internal or external).

- No implicit trust for insiders, VPN users, or cloud workloads.

4. **Proactive Threat Hunting Over Reactive Alerting**
   - Hypothesis-driven hunts assume adversaries are hiding and actively search for signs of compromise.
   - Baseline "normal" behavior so anomalies stand out.

5. **Resilience and Recovery Planning**
   - Assume you'll need to isolate, eradicate, and restore—plan for it.
   - Test backups, incident playbooks, and communication under breach conditions.

Blue Team Tech Stack

The **Blue Team tech stack** refers to the collection of tools, platforms, and technologies that defensive security teams (SOC analysts, incident responders, threat hunters, detection engineers, etc.) use daily to monitor, detect, analyze, respond to, and hunt for threats. In 2026, the stack has evolved significantly toward **integrated, AI-augmented platforms** that reduce manual effort, correlate signals across domains, and support proactive operations under the Assume Breach mindset.

Modern Blue Team stacks are rarely "one-tool-fits-all." Organizations blend **commercial enterprise solutions** (for scalability, support, and AI capabilities) with **open-source tools** (for flexibility, cost, and customization). The exact mix depends on company size, budget, cloud vs. on-prem footprint, and whether the SOC is in-house, co-managed, or outsourced (MDR).

Core Layers of a Modern Blue Team Tech Stack (2026):

1. **Endpoint Detection and Response (EDR) / Extended Detection and Response (XDR)**
   - The foundation for endpoint visibility and behavioral detection.
   - These tools monitor processes, file changes, registry activity, network connections, and memory for anomalies (e.g., LOLBin abuse, credential dumping, ransomware prep).
   - Top commercial platforms:
     - **CrowdStrike Falcon** — AI-driven behavioral analytics, rapid response, strong MITRE ATT&CK coverage, and Falcon OverWatch (human-augmented hunting).

        - **Microsoft Defender XDR** — Unified across endpoints, identity (Entra ID), email, cloud (Azure/365), and often the default for Microsoft-heavy environments.
        - **SentinelOne Singularity** — Autonomous AI, rollback capabilities, strong against fileless attacks.
        - Others: Palo Alto Cortex XDR, Trend Micro Vision One, Sophos XDR.
    - Many now include built-in **SOAR-like automation** for containment (isolate endpoint, kill process, block IP).

2. **SIEM (Security Information and Event Management)**
    - Central log aggregation, correlation, alerting, and historical search platform.
    - Ingests logs from endpoints, network, cloud, firewalls, AD, applications → runs rules/queries for detection and hunting.
    - Popular options:
        - **Splunk Enterprise Security** — Industry standard for complex queries (SPL), dashboards, and correlation.
        - **Microsoft Sentinel** — Cloud-native, cost-effective for Azure shops, tight Defender XDR integration.
        - **Elastic Stack (ELK: Elasticsearch, Logstash, Kibana)** — Open-source powerhouse, widely used for custom SIEM + observability.
        - Others: IBM QRadar, Google Chronicle, Sumo Logic, Graylog.
    - Modern SIEMs emphasize **data lake** architecture, UEBA (User/Entity Behavior Analytics), and AI for anomaly detection.

3. **Network Security Monitoring & Intrusion Detection/Prevention (NIDS/NIPS)**
    - Deep packet inspection and traffic analysis to catch C2, exfiltration, lateral movement.
    - Key tools:
        - **Suricata** — High-performance, multi-threaded open-source IDS/IPS (rules like Snort but faster).
        - **Zeek (formerly Bro)** — Scriptable network analysis framework for protocol metadata and anomaly detection.
        - **Snort** — Classic open-source IDS, still widely deployed.
        - **Wireshark** — Essential for manual PCAP analysis (used in your course capstone!).
        - Commercial NDR (Network Detection and Response): Darktrace, Vectra AI, ExtraHop.

4. **Threat Intelligence Platforms & IOC Enrichment**
    - Integrate feeds (CISA alerts, AlienVault OTX, abuse.ch, MISP) to enrich alerts and enable retroactive hunting.

- Tools: **MISP** (open-source sharing), ThreatConnect, Recorded Future, Anomali, or built-in features in XDR/SIEM.

5. **Incident Response & Case Management**
   - Ticketing, playbook execution, collaboration.
   - **TheHive** + **Cortex** (open-source IR platform + analyzer).
   - Commercial: Splunk SOAR, Palo Alto Cortex XSOAR, Swimlane, Torq.

6. **Vulnerability Management & Scanning**
   - Identify and prioritize weaknesses before exploitation.
   - **Tenable Nessus**, Qualys, Rapid7 InsightVM, OpenVAS (open-source).

7. **Endpoint & System Forensics Tools**
   - **Sysmon** (Microsoft) → Critical for process/network logging (heavily used in your course exercises).
   - **Velociraptor** or **Osquery** — Live response and endpoint querying.
   - Memory: **Volatility**; Disk: **Autopsy**, **Arsenal Image Mounter**.

8. **Threat Hunting & Detection Engineering Tools**
   - **Sigma** rules (universal detection format — central to your course!).
   - **YARA** for file-based hunting.
   - **MITRE ATT&CK Navigator** — Mapping and visualization (used throughout your course).
   - Query languages: Splunk SPL, KQL (Elastic/Sentinel), custom scripts.

9. **Other Supporting Tools**
   - **CyberChef** — Data manipulation (decoding, hashing — great for IOC extraction).
   - **NetworkMiner** — PCAP parsing for artifacts.
   - Cloud-specific: AWS GuardDuty, Azure Defender, GCP Security Command Center.
   - Honeypots (e.g., Cowrie, Dionaea) for early warning.

Incident Response

The **Incident Response Process** is a structured, repeatable methodology organizations use to handle cybersecurity incidents—from initial detection through full recovery and improvement. It minimizes damage, reduces recovery time and costs, preserves evidence, and strengthens future defenses.

This aligns directly with the **Assume Breach** mindset: incidents are inevitable, so focus on rapid, effective handling.

Two of the most widely adopted frameworks in 2026 are:

- **NIST SP 800-61 Revision 3** (released April 2025) — aligned with NIST Cybersecurity Framework (CSF) 2.0, emphasizing integration into broader risk management with a focus on **Detect**, **Respond**, and **Recover** functions (building on foundational **Govern**, **Identify**, and **Protect** activities).

- **SANS Institute** model — a practical, six-step tactical process commonly taught in blue team training.

NIST SP 800-61 Rev. 3 Incident Response Model (2025 Update)
NIST's latest revision shifts from a rigid four-phase lifecycle to a more flexible, CSF 2.0-integrated approach. It treats incident response as an ongoing capability embedded in risk management, with continuous improvement. Core incident handling focuses on three top-level functions:

1. **Detect** Identify potential cybersecurity events and determine if they constitute an incident.
    - Activities: Continuous monitoring (SIEM, EDR, logs, network traffic), anomaly detection, alert triage, initial validation (true/false positive), scoping (is this isolated or widespread?).
    - Goal: Quick confirmation and classification (e.g., ransomware prep, credential access, exfiltration).
    - Tools/Techniques: SIEM queries, EDR behavioral alerts, threat intel enrichment.

2. **Respond** Take actions to contain the incident, eradicate the threat, and begin recovery.
    - Sub-activities:
        - **Containment** — Stop spread (isolate endpoints, block C2, disable accounts, network segmentation). Short-term vs. long-term containment.
        - **Eradication** — Remove root cause (delete malware, close vulnerabilities, revoke compromised credentials, clean persistence mechanisms like scheduled tasks or registry keys).
        - **Recovery** — Restore systems to normal, validate integrity, monitor for re-infection.
    - Goal: Limit blast radius and return to operations safely.

3. **Recover** (with overarching **Post-Incident Activity**) Restore capabilities fully while improving resilience.
    - Activities: System restoration from backups, business continuity testing, monitoring for recurrence, root cause analysis.
    - **Post-Incident / Lessons Learned**: Conduct after-action review (AAR), update playbooks, detection rules, and policies; share IOCs; measure effectiveness (dwell time, impact).
    - Goal: Turn every incident into a defensive.

NIST emphasizes preparation (via Govern/Identify/Protect) happens continuously outside active incidents, and improvement loops back via the Identify function.

SANS Institute Six-Step Model
This tactical, hands-on model is popular in SOC/Blue Team training and certification paths (e.g., GIAC GCIH).

1. **Preparation** Build capabilities before incidents occur: IR plan, team roles (including legal/PR/HR), tools (EDR, SIEM, forensics kits), playbooks, training, backups tested.

2. **Identification** Detect and confirm an incident (similar to NIST Detect): triage alerts, analyze logs/PCAPs, classify severity, determine scope.

3. **Containment** Isolate affected systems to prevent further damage (short-term: kill processes/isolate; long-term: rebuild/re-image).

4. **Eradication** Remove the threat actor's artifacts (malware, backdoors, persistence, compromised accounts).

5. **Recovery** Return systems to production, monitor closely, validate no re-compromise.

6. **Lessons Learned** Post-incident review: what went well, gaps, metrics (MTTD/MTTR), updates to detections/playbooks, executive reporting.

Why This Process Matters in Blue Teaming

- Reduces **dwell time** (attacker time inside) from weeks/months to hours/days.

- Limits financial/reputational damage (e.g., ransomware encryption prevented by early containment).

- Generates **actionable intelligence** (IOCs, TTPs) to improve detections.
- Builds **portfolio artifacts** — incident reports, timelines, ATT&CK mappings.

Mastering this process turns reactive firefighting into proactive defense.

Metrics that Matter

Blue teams (SOC analysts, incident responders, threat hunters, and detection engineers) measure success through **key performance indicators (KPIs)** and metrics that focus on **speed**, **accuracy**, **efficiency**, and **overall defensive effectiveness**. These metrics help quantify how well the team prevents, detects, contains, and recovers from threats—aligning directly with the **Assume Breach** mindset and frameworks like NIST and SANS incident response.

The most critical metrics emphasize reducing attacker **dwell time** (how long threats go undetected) and minimizing business impact. Here's a brief overview of the ones that matter most in 2026:

Core Time-Based Metrics (The "Big Three" for Most Teams)
These track speed across the incident lifecycle and are the most commonly reported to leadership.

- **Mean Time to Detect (MTTD)** Average time from when malicious activity begins to when it's first detected/identified.
    - Why it matters: Lower MTTD means threats are caught earlier, reducing dwell time and potential damage (e.g., before lateral movement or exfiltration).
    - Good benchmark: Top teams aim for minutes to hours (e.g., <4 hours); many still struggle with days/weeks.
    - Ties to: Detection engineering, SIEM/EDR tuning, threat.

- **Mean Time to Respond / Contain (MTTR or MTTC)** Average time from detection to containment (stopping spread, e.g., isolating endpoints, blocking C2) or full remediation/recovery.
    - Why it matters: Fast response limits blast radius—critical for ransomware or APTs.
    - Good benchmark: Hours for critical incidents; top performers target <1 hour for containment.

    - Ties to: Alert triage, playbooks, automation (your capstone's containment recommendations).

- **Dwell Time** Total time an attacker remains undetected in the environment (from initial compromise to detection or eviction).
    - Why it matters: The ultimate outcome metric—shorter dwell = better overall defense. Often combines MTTD + time to full eradication.
    - Good benchmark: Industry averages have dropped but still often weeks; elite teams aim for hours/days.

Accuracy & Quality Metrics
These ensure the team isn't overwhelmed by noise and focuses on real threats.

- **False Positive Rate (FPR)** Percentage of alerts investigated that turn out benign.
    - Why it matters: High FPR burns analyst time and causes alert fatigue.
    - Goal: Keep low (e.g., <20-30%) through rule.

- **True Positive Rate / Detection Rate** Percentage of alerts that confirm genuine incidents, or percentage of known attack techniques your detections catch.
    - Why it matters: Measures signal quality and coverage.

- **Alert-to-Incident Conversion Rate** Percentage of alerts that become confirmed incidents (vs. noise).
    - Why it matters: Shows detection effectiveness beyond volume.

Coverage & Maturity Metrics
These assess defensive breadth and proactive capability.

- **Detection Coverage (e.g., MITRE ATT&CK)** Percentage of ATT&CK techniques/sub-techniques with visibility, logging, or active detections (endpoint, network, cloud layers).
    - Why it matters: Identifies blind.
    - Goal: High coverage (e.g., 70-90%+) on high-impact techniques.

- **Threat Hunting Effectiveness** Number/frequency of proactive hunts that find hidden threats (or % of incidents found via hunting vs. reactive alerts).
    - Why it matters: Shows maturity beyond reactive alerting.

Other Practical Metrics

- **Mean Time to Acknowledge (MTTA)** — Time from alert to first analyst action (triage speed).
- **Incident Closure Rate / Resolution Rate** — % of incidents closed within SLA.
- **Escalation Rate** — % of alerts needing Tier 2/3 help.
- **Analyst Utilization / Case Load** — Hours per analyst on investigations (balance burnout vs. coverage).

Why Blue Team Matters

In today's threat landscape — where ransomware, nation-state actors, and commodity malware constantly evolve — the Blue Team is the organization's primary line of defense. Effective blue teaming reduces dwell time (how long attackers remain undetected), limits damage from breaches and builds resilience. Organizations with strong Blue Team capabilities detect threats faster, respond more effectively, and recover with less disruption.

This course, *Blue Team Fundamentals: Practical Defense & Detection*, is designed to immerse you in real SOC workflows. Through hands-on exercises, you'll build attacker awareness to think like the Red Team while deciding and acting like a defender. You'll create tangible deliverables—a personal Defensive Playbook, a Detection Rule Library, threat hunting queries, incident reports, and more—that demonstrate practical skills employers value.

By the end, you'll have experienced what it's like to operate in a real Security Operations Center: triaging alerts under time pressure, hunting hidden threats, analyzing network captures, mapping attacks to MITRE ATT&CK, and documenting findings professionally.

Welcome to the Blue Team. Let's get to work defending what matters.

Module 1: The Defensive Mindset

Blue teamwork starts with perspective.

Attackers only need one path to succeed, but defenders can win by creating visibility, slowing the attacker down, and responding before the damage becomes widespread.

In a real SOC, analysts rarely begin with full context. They begin with a suspicious email, a strange process, a failed login pattern, or a user report. The defensive mindset is the habit of asking: *What am I seeing, why does it matter, and what should I check next*?

This module introduces a practical way to think about attacks as sequences instead of isolated events. Phishing, execution, credential theft, lateral movement, and impact all leave different traces. Good defenders learn to recognize those traces early and connect them into a story.

Students should leave this section understanding that prevention is important, but visibility and response are just as important. Mature defenders assume that some controls will fail. Their job is to make sure a single mistake does not become a business-wide incident.

As you move through the course, keep returning to four questions:

- What would this look like in logs?
- Which control should have stopped it?
- How would I contain it?
- How would I explain it clearly to someone else?

Those questions form the foundation of effective blue teamwork.

By the end of this module, students should be comfortable thinking in terms of detection opportunities, response priorities, and attack progression rather than only individual tools or alerts.

Exercise 1: Attack Path Analysis

Purpose

Students will analyze a realistic phishing-based ransomware attack, map it across the **Cyber Kill Chain** and **MITRE ATT&CK**, and identify **7+ detection and prevention opportunities** across people, process, and technology.

Learning Objectives

By the end of this exercise, students will be able to:

1. Explain how a phishing email can lead to full ransomware deployment
2. Map attacker actions to:
    - Cyber Kill Chain phases
    - MITRE ATT&CK tactics and techniques
3. Identify **at least 7 detection or disruption points**
4. Recommend **realistic security controls** (technical + procedural)
5. Communicate attack paths clearly to both technical and non-technical audiences

Scenario Summary (Read Carefully)

You are part of the security team for a **150-person professional services company** using:

- Microsoft 365 (Exchange Online, SharePoint)
- Windows 10 endpoints
- On-prem file server
- No dedicated SOC

An employee in **Finance** receives an email appearing to come from a known vendor with an attached invoice. Two days later, multiple systems are encrypted and a ransomware note appears.

There are no files to analyze in this exercise. Using only the information above **reconstruct the attack path**, map it to industry frameworks, and identify **realistic detection or disruption opportunities**.

Tasks

Step 1 – Map the Attack Chain

Using the information provided, map the attack from start to finish:

1. Initial phishing email
2. User interaction
3. Malware execution

4. Credential access
5. Lateral movement
6. Ransomware deployment

Step 2 – Cyber Kill Chain Mapping

For each step, identify the corresponding **Cyber Kill Chain phase**:

1. Reconnaissance
2. Weaponization
3. Delivery
4. Exploitation
5. Installation
6. Command & Control
7. Actions on Objectives

Step 3 – MITRE ATT&CK Mapping

For each attack step, map **at least one MITRE technique**.

Step 4 – Identify Detection Points (Minimum 7)

For each phase, answer:
"What could realistically detect or disrupt this?"
Detection can include:

- Technical controls
- User behavior
- Logging and monitoring
- Policy/process

Each detection point should include:

- **What detects it**
- **Why it works**
- **Who would see the alert**

Step 5 – Prioritize

Rank your detection points:

- **High impact / low effort**
- **High impact / high effort**
- **Low impact**

Module 2: MITRE ATT&CK for Defenders

MITRE ATT&CK gives defenders a common language for describing adversary behavior. Instead of saying only that something looks suspicious, analysts can describe the tactic, technique, and likely objective behind the activity.

This framework is useful because it organizes attacker behavior into observable categories such as Initial Access, Execution, Persistence, Credential Access, Lateral Movement, and Exfiltration. That structure helps defenders map what happened, decide what data sources are needed, and identify gaps in monitoring.

For entry-level analysts, ATT&CK is not just a research tool. It is a daily operational aid. It helps with triage, alert investigation, threat hunting, reporting, and communication with other analysts. When a student can say, “This looks like T1059 PowerShell execution followed by T1003 credential dumping,” they are already communicating at a more professional level.

This module also introduces ATT&CK Navigator, which helps visualize techniques, highlight priorities, and document defensive coverage. Students should think of it as a practical way to show what an adversary did and what the organization can currently detect.

The goal is not to memorize every technique ID. The goal is to learn how to use ATT&CK to ask better questions: What is the attacker trying to accomplish? What evidence would reveal that behavior? Where do we have coverage, and where are we blind?

By the end of this module, students should be able to translate raw security activity into an ATT&CK-informed explanation that supports detection, hunting, and incident response.

Framework	What It Provides	Relationship to ATT&CK
Cyber Kill Chain	7 linear phases	Broader view; ATT&CK provides granular techniques within each phase
NIST CSF	Security program functions (Identify, Protect, Detect, Respond, Recover)	ATT&CK informs the "Detect" function
CEH	Attacker methodology	ATT&CK describes these attacks from defender's view
MITRE D3FEND	Defensive countermeasures	Sister framework - maps defenses to ATT&CK techniques

Tactics are your '**why**,' techniques are your '**what**,' sub-techniques are your '**how**.' As defenders, we detect techniques but understand them through the lens of tactics.

Remember the attack path analysis from Exercise #1? That attack moved through these exact tactics. ATT&CK gives us standardized language to describe what we saw

Think of it as 'painting' the threat actor's TTP portrait. This visual becomes your detection priority list.

Exercise 2: MITRE ATT&CK for Defenders

Learning Objectives

By the end of this exercise, students will be able to:

1. Navigate and utilize the MITRE ATT&CK Navigator interface
2. Map real-world threat actor campaigns to ATT&CK techniques
3. Identify relevant detection data sources for specific techniques
4. Understand the full attack lifecycle from Initial Access to Exfiltration
5. Create shareable threat intelligence visualizations

Required Resources

- MITRE ATT&CK Navigator: https://mitre-attack.github.io/attack-navigator/
- MITRE ATT&CK Website: https://attack.mitre.org/
- APT29 Group Profile: https://attack.mitre.org/groups/G0016/

Background – APT29 (Cozy Bear/The Dukes)

APT29 is a sophisticated threat actor group attributed to the Russian Foreign Intelligence Service (SVR). Known for:

- **Active Since:** At least 2008
- **Target Profile:** Government agencies, think tanks, healthcare, energy sectors
- **Notable Campaigns:** Democratic National Committee breach (2016), SolarWinds supply chain attack (2020)
- **Characteristics:** Stealthy, patient, sophisticated tradecraft

Why Study APT29?

- Well-documented tactics and techniques
- Represents nation-state level threats
- Multiple campaigns provide diverse technique examples
- Excellent case study for defense planning

Part 1: Setup

Step 1: Access ATT&CK Navigator

1. Open your web browser
2. Navigate to: https://mitre-attack.github.io/attack-navigator/

3. Click **"Create New Layer"**
4. Select **"Enterprise ATT&CK v15"** (or latest version)

Step 2: Configure Your Layer

1. Click the **layer controls** (gear icon, top right)
2. Set layer name: "APT29 Campaign Analysis - [Your Name]"
3. Set description: "Mapping APT29 tactics from Initial Access to Exfiltration"
4. Click **Save**

Step 3: Familiarize Yourself with the Interface

- **Left Panel:** Tactic categories (columns)
- **Center Panel:** Techniques organized by tactic
- **Top Menu:** Tools for selection, annotation, and export
- **Search Function:** Quickly find specific techniques

Part 2: Research APT29 Techniques

Step 1: Access APT29 Group Profile

1. Open a new browser tab
2. Navigate to: https://attack.mitre.org/groups/G0016/
3. Review the **"Techniques Used"** section
4. Note: MITRE documents techniques APT29 has demonstrated across multiple campaigns

Step 2: Organize Your Research

Create a table (use your preferred note-taking tool) with these columns:

- Tactic
- Technique ID
- Technique Name
- Data Sources for Detection
- Campaign/Notes

Part 3: Map Techniques in Navigator

Your mission: Identify and map at least **15 techniques** spanning from **Initial Access** through **Exfiltration**. You must include techniques from these minimum tactics:

- Initial Access (at least 1)

- Execution (at least 2)
- Persistence (at least 2)
- Privilege Escalation (at least 1)
- Defense Evasion (at least 2)
- Credential Access (at least 2)
- Discovery (at least 1)
- Lateral Movement (at least 1)
- Collection (at least 1)
- Command and Control (at least 1)
- Exfiltration (at least 1)

Example to Get You Started:

Tactic	Technique ID	Technique Name
Initial Access	T1566.001	Phishing: Spearphishing Attachment
Initial Access	T1566.002	Phishing: Spearphishing Link
Execution	T1059.001	Command and Scripting Interpreter: PowerShell
Persistence	T1053.005	Scheduled Task/Job: Scheduled Task
Defense Evasion	T1027	Obfuscated Files or Information

Mapping Process:

For each technique you identify:

1. **Find the technique in Navigator**
    - Use the search function or browse the tactic column
    - Click on the technique box
2. **Select and annotate**
    - Click the technique to highlight it
    - In the technique detail panel (right side):
        - Set color (suggest: red/orange for high priority)
        - Add score if desired (1-100)
        - Add comment with relevant campaign info
3. **Document detection data sources**
    - Click the technique name to open its ATT&CK page
    - Scroll to **"Detection"** section
    - Note the **Data Sources** and **Data Components**

- Record these in your research table

Part 4: Analysis and Documentation (15 minutes)

Step 1: Export Your Layer

1. Click **layer controls** → **download layer**
2. Save as JSON file: "APT29_Analysis_[YourName].json"

Step 2: Generate Screenshot

1. Use browser screenshot tool or snipping tool
2. Capture your completed Navigator layer
3. Save as: "APT29_Navigator_[YourName].png"

Step 3: Complete Analysis Report

Answer these questions:

1. **What was the most common tactic used by APT29 in your analysis?**
2. **Which technique did you find most challenging to detect? Why?**
3. **Identify 3 data sources that appear repeatedly across multiple techniques:**
    - Data Source 1:
    - Data Source 2:
    - Data Source 3:
4. **If you were a defender, which 3 techniques would you prioritize for detection? Justify your choices.**
5. **What patterns do you notice in APT29's progression from Initial Access to Exfiltration?**

TIPS FOR SUCCESS

- **Use Multiple Sources:** Don't rely solely on the ATT&CK group page. Check:
    - Public threat intelligence reports
    - MITRE's "Software" pages for tools APT29 uses
    - Campaign-specific write-ups
- **Focus on Detection:** The goal isn't just to list techniques, but to understand HOW to detect them
- **Think Like a Defender:** Consider which techniques would be easiest/hardest to detect in YOUR environment

- **Leverage Sub-techniques:** Many techniques have sub-techniques that provide more granular details
- **Color Coding Strategy:** Consider using different colors for:
    - Techniques with strong detection capabilities (green)
    - Techniques difficult to detect (red)
    - Medium difficulty (yellow/orange)

Module 3: Windows Event Log Analysis

Windows systems generate a large amount of security-relevant telemetry. For blue team analysts, event logs are one of the most important sources of evidence because they record authentication activity, privilege use, process creation, service installation, PowerShell execution, and many other behaviors tied to attacker tradecraft.

This module introduces students to the idea that logs are not just technical records. They are evidence. An effective analyst uses them to confirm what happened, when it happened, who performed the action, and what the likely impact was.

Students will focus on the event types most commonly used in investigations: Security logs for logons and privilege activity, Sysmon for process and network visibility, and PowerShell logging for high-value command evidence. The goal is not to read everything, but to filter toward the events most likely to answer the investigation question.

A key beginner skill is learning the difference between normal administrative activity and suspicious behavior. A logon event is not automatically bad. A PowerShell process is not automatically malicious. Context, timing, user account, host role, and related events determine whether something is normal or dangerous.

This section also reinforces correlation. Rarely does one event tell the whole story. A successful investigation often comes from linking a process event to a privilege event, then linking both to a script block or a network connection. That is how defenders move from isolated evidence to a complete attack timeline.

By the end of this module, students should be able to approach Windows logs with a repeatable method: filter, correlate, interpret, and document.

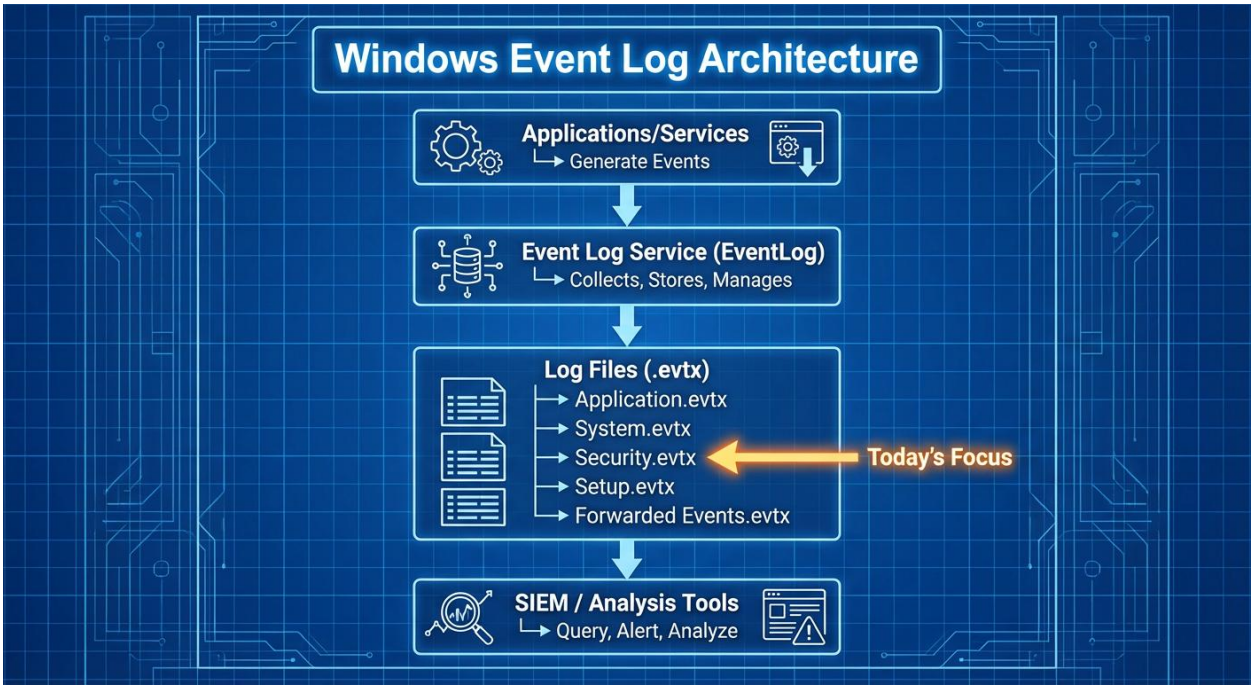
Event Log Structure

Every Windows event has the same basic structure:

TimeCreated:	When it happened
EventID:	What type of event
Computer:	Which system
Provider:	What generated it
Message:	Detailed description
Level:	Info/Warning/Error

The Two Most Important Fields:

1. **Event ID** - A number that identifies the event type (e.g., 4624 = successful logon)
2. **TimeCreated** - The timestamp showing when it happened (critical for building timelines)



1.0 Windows Security Event Log

What It Records

The Windows Security Event Log captures **security-sensitive actions**:

- User logins and logouts
- Privilege usage (like administrator rights)
- Process creation
- File and registry access
- Account management

Critical Security Event IDs for This Exercise

Event ID	Event Name	What It Means	Why It Matters
4624	Successful Logon	User authenticated successfully	Shows who logged in and when

Event ID	Event Name	What It Means	Why It Matters
4672	Special Privileges Assigned	User granted elevated rights	**Detects privilege escalation**
4673	Sensitive Privilege Used	Elevated privilege was exercised	Shows what privileged action occurred
4688	Process Created	Program/command was executed	Shows what the attacker ran

Understanding Event 4672: Special Privileges

Event 4672 is your **red flag for privilege escalation**. When this event fires, Windows is telling you: *"I just gave this user powerful capabilities."*

Prime Example: SeDebugPrivilege

Key Privilege: SeDebugPrivilege
What it does: Allows reading ANY process's memory, bypassing normal security boundaries
Normal use: Only SYSTEM account and administrators should have this
Attacker use: Required to dump credentials from LSASS (credential theft)

Understanding Event 4624: Logon Events

Event 4624 shows successful authentication. The **Logon Type** tells you HOW the user logged in:

Logon Type	Name	Example	Normal?
2	Interactive	User at keyboard	Yes
3	Network	Accessing file share	Yes
4	Batch	Scheduled task	Verify legitimacy
5	Service	Windows service	Verify legitimacy
10	RemoteInteractive	RDP session	Check source IP

What to look for:

- Unusual logon times (2 AM on Sunday?)
- Logon Type 10 from unknown IPs (unauthorized RDP)
- Multiple failed logons followed by success (brute force)

2.0 Sysmon Event Log

What Is Sysmon?

Sysmon (System Monitor) is a free Windows service from Microsoft that provides **enhanced logging** capabilities beyond what Windows natively records.

Why It Exists:

Native Windows logging doesn't capture:

- Full command-line arguments
- Network connections by processes
- DLL loading activity
- **Process memory access** ← Critical for detecting credential theft!

Sysmon fills these gaps.

Critical Sysmon Event IDs for This Exercise

Event ID	Event Type	What It Logs	Detection Use Case
1	Process Creation	Full command line with arguments	Detect malicious commands
3	Network Connection	Process → IP:Port	Detect C2 communication
10	Process Access	One process accessing another's memory	**Detect LSASS credential dumping**
11	File Created	File written to disk	Detect malware staging

Event 10 is the star of today's exercise. It fires when one process opens a handle to another process's memory space.

Key Fields from Sysmon:

Field	Description	What to Look For
SourceImage	Process that accessed memory	PowerShell.exe, cmd.exe, rundll32.exe = suspicious
TargetImage	Process being accessed	lsass.exe = high-value target
GrantedAccess	Permission level requested	0x1010, 0x1410, 0x1438 = memory read (dangerous)
UtcTime	When it happened	Use for timeline correlation

GrantedAccess Codes Explained:

Code	Permission	Meaning	Risk Level
0x1000	QUERY_INFORMATION	Read basic process info	Low
0x1010	QUERY_INFORMATION + VM_READ	Read process memory	**HIGH**
0x1400	QUERY_LIMITED_INFORMATION	Limited info read	Low
0x1410	QUERY_LIMITED_INFORMATION + VM_READ	Read memory (limited)	**HIGH**
0x1438	Multiple read permissions	Full memory access	**CRITICAL**

What Is LSASS and Why Does It Matter?

LSASS (Local Security Authority Subsystem Service) is a Windows process that:

- Handles user authentication
- Stores credentials in memory (passwords, hashes, Kerberos tickets)
- Location: C:\Windows\System32\lsass.exe
- Always running on every Windows system

Dumping LSASS memory gives attackers:

- User passwords in plaintext (on older systems)
- NTLM password hashes
- Kerberos authentication tickets
- Security tokens

What Attackers Do Next:

1. Steal credentials from LSASS
2. Use stolen credentials to access other systems (lateral movement)
3. Escalate to Domain Admin (if credentials were found)
4. Compromise entire network

On a Domain Controller (like in your exercise):

- LSASS contains credentials for ALL domain users
- Compromising DC LSASS = entire domain compromise
- This is why it's a CRITICAL severity incident

How to Detect:

1. **Sysmon Event 10** - Process accessing lsass.exe with read permissions (0x1010, 0x1410, 0x1438)
2. **Security Event 4672/4673** - SeDebugPrivilege usage
3. **PowerShell Event 4104** - Script blocks containing Mimikatz, sekurlsa, logonpasswords
4. **Unusual parent-child relationships** - cmd.exe → powershell.exe → accesses lsass.exe

3.0 PowerShell Event Log

Why PowerShell Logging Matters
PowerShell is the most powerful automation tool in Windows—and attackers love it because:

- Installed by default on every Windows system
- Direct access to Windows APIs and .NET Framework
- Can execute code in memory (no files on disk)
- Often trusted by security tools (signed by Microsoft)

The Good News: PowerShell has robust logging that captures everything attackers do.

Critical PowerShell Event IDs for This Exercise:

Event ID	Event Type	What It Logs	Value
4104	Script Block Logging	**Full PowerShell script text**	**HIGHEST**
800	Pipeline Execution	Command pipeline	Medium
4103	Module Logging	Cmdlets executed	Medium

Understanding Event 4104: Script Block Logging
Event 4104 is your goldmine. It captures the actual PowerShell code being executed, even if:

- The command was encoded (base64)
- The script was run from memory
- The attacker tried to obfuscate it

Recognizing PowerShell Attack Indicators

Red Flag Command-Line Arguments:

Argument	What It Does	Why Attackers Use It
-enc or -EncodedCommand	Base64-encoded PowerShell	Evade signature detection
-noP or -NoProfile	Skip loading profile scripts	Avoid detection scripts
-w hidden or -WindowStyle Hidden	Hide PowerShell window	Stealth
-ExecutionPolicy Bypass	Disable script restrictions	Run unsigned scripts
-noni or -NonInteractive	No user prompts	Automated execution

Red Flag Keywords in Script Blocks:

Keyword	What It Indicates
Invoke-Mimikatz	Credential dumping tool
sekurlsa::logonpasswords	Mimikatz command to extract passwords
Invoke-Expression or IEX	Execute code dynamically (common in attacks)

Keyword	What It Indicates
DownloadString	Download code from remote server
Net.WebClient	Network download capability
Invoke-Shellcode	Execute shellcode in memory
-enc followed by long base64	Encoded command (obfuscation)

PowerShell Event 4104: What You'll See

Event 4104 captures the decoded script, even if the attacker encoded it. This means:

Attacker runs:

```
powershell.exe -enc SQBuAHYAbwBrAGUALQBNAGkAbQBpAGsAYQB0AHoA...
```

Event 1404 shows:

```
Creating Scriptblock text (1 of 1):
Invoke-Mimikatz -Command "sekurlsa::logonpasswords"
```

Log Analysis Methodology

You can't read every event. You need a systematic approach.

The FOCUS Framework

Use this 5-phase methodology to find the needle in the haystack:

F - Filter for High-Fidelity Alerts
O - Observe Temporal Patterns
C - Correlate Across Log Sources
U - Understand Context
S - Systematize with Timeline

Phase 1: FILTER for High-Fidelity Alerts

Strategy: Start with events that are almost always suspicious

High-Fidelity Event IDs:

- Sysmon Event 10: PowerShell → LSASS
- Security Event 4672: SeDebugPrivilege for non-admin users

- PowerShell Event 4104: Scripts containing "Invoke-Mimikatz"

Practical Steps:

1. Open CSV file in Excel
2. Apply **AutoFilter** (Data → Filter)
3. Filter EventID column to show only Event 10
4. Further filter to show only suspicious combinations

Phase 2: OBSERVE Temporal Patterns

Strategy: Look for time-based anomalies

Questions to Ask:

- Is activity happening outside business hours?
- Are multiple related events within seconds of each other?
- Is there a sudden spike in event volume?

Time Correlation Window:
Events within **1-2 seconds** of each other are likely related:

Phase 3: CORRELATE Across Log Sources

Strategy: Connect the dots between different logs

Phase 4: UNDERSTAND Context

Strategy: Assess impact and prioritize response

Questions to Ask:

- What systems role?
- What user account?
- What data access?
- Normal user behavior?

Phase 5: SYSTEMATIZE with Timeline

Strategy: Build a chronological attack sequence

Why Timelines Matter:

- Shows attack progression
- Proves causation (not just correlation)

- Identifies attacker dwell time
- Required for incident reports
- Supports forensic investigation

Work Smart, Not Hard

- Filter first, read second
- Search for keywords, don't browse
- Focus on high-fidelity events

Exercise #3: Detect Credential Dumping with Mimikatz

Objectives

By the end of this exercise, you will be able to:

- Detect credential dumping attacks in Windows event logs
- Correlate evidence across Security, Sysmon, and PowerShell logs
- Timeline an attack from initial execution through credential theft
- Identify PowerShell Empire C2 framework indicators
- Map attack behaviors to MITRE ATT&CK framework
- Extract Indicators of Compromise (IOCs) for threat hunting

Required Resources

- Three CSV files
  - PowerShell_Events_parsed.csv
  - Security_Events_Parsed.csv
  - Sysmon_Events_Parsed.csv
- Microsoft Excel or other spreadsheet application

The Scenario

Security Alert
Date/Time: August 7, 2020, 14:30:00 UTC
Alert Source: Sysmon Process Access Detection
Priority: CRITICAL
Alert Description: "Suspicious LSASS process memory access detected"
System Information:

- **Hostname:** MORDORDC.THESHIRE.LOCAL
- **Role:** Windows Server 2019 - Domain Controller
- **Domain:** THESHIRE.LOCAL
- **Compromised User:** pgustavo (Domain User Account)
- **IP Address:** Internal network

Initial Triage Findings

Your Tier 1 analyst escalated this alert after confirming:

- PowerShell.exe accessed LSASS.exe memory (Sysmon Event ID 10)
- SeDebugPrivilege was used by PowerShell process
- Suspicious encoded PowerShell commands detected
- **TARGET IS A DOMAIN CONTROLLER** - High Impact!

Your Mission

You are the **Tier 2 SOC Analyst** assigned to this incident. Your job:

1. **Investigate** - Determine what happened, how, and when
2. **Timeline** - Build a complete attack sequence with timestamps
3. **Identify** - Extract all Indicators of Compromise (IOCs)
4. **Classify** - Map to MITRE ATT&CK framework
5. **Recommend** - Provide containment and remediation actions

Evidence Files Provided

You have been given three CSV log files extracted from MORDORDC covering a 10-minute critical window:

1. Sysmon_Events_Parsed.csv

- **Source:** Sysmon Enhanced Logging
- **Events:** 679 total events
- **Key Event IDs:**
  - **Event 1** - Process Creation (with CommandLine)
  - **Event 3** - Network Connection
  - **Event 10** - Process Access (**THE SMOKING GUN**)
  - **Event 11** - File Creation
- **Parsed Columns:** SourceImage, TargetImage, GrantedAccess, CommandLine, User
- **Why This Matters:** Sysmon Event 10 detects when one process accesses another process's memory - the hallmark of credential dumping

2. Security_Events_Parsed.csv

- **Source:** Windows Security Event Log
- **Events:** 702 total events
- **Key Event IDs:**
  - **Event 4624** - Account Logon
  - **Event 4672** - Special Privileges Assigned (**SeDebugPrivilege**)
  - **Event 4673** - Sensitive Privilege Use
  - **Event 4688** - Process Creation
- **Parsed Columns:** SubjectUserName, Privileges, NewProcessName, CommandLine
- **Why This Matters:** Shows when SeDebugPrivilege (needed to access LSASS) was granted

3. PowerShell_Events.csv

- **Source:** PowerShell Script Block Logging
- **Events:** 4,644 total events (large!)

- **Key Event IDs:**
  - **Event 800** - Pipeline Execution
  - **Event 4104** - Script Block Logging (**Shows actual commands**)
- **Format:** Message column contains full PowerShell script text
- **Why This Matters:** Reveals what commands the attacker executed, including Invoke-Mimikatz

Important Notes:

1. **TimeCreated column is empty** - The actual timestamp is in the Message column as "UtcTime"
2. **Most data is in the Message column** - You'll need to read the Message text to find details
3. **Multi-line messages** - Excel may display these across multiple visible lines

Investigation Instructions

Phase 1: Find the LSASS Memory Access (15 minutes)

Step 1: Open Sysmon_Events_Parsed.csv

1. Open Sysmon_Events_Parsed.csv in Excel
2. Enable **AutoFilter** (Data tab → Filter)
3. **Filter EventID column** to show only: **10**
4. You should now see **272 Process Access events**

Step 2: Find LSASS Access

5. **Filter the TargetImage column** to show only rows containing: **lsass**
   - In Excel: Click dropdown → Text Filters → Contains → Type "lsass"
6. **Expected Result:** You should find **7 events** where a process accessed LSASS memory

Step 3: Identify Suspicious Access

Look at the **SourceImage** column. Normal processes that access LSASS:

- svchost.exe - Windows service host (usually safe)
- services.exe - Windows Services Control Manager (safe)
- **powershell.exe** - SUSPICIOUS! PowerShell shouldn't access LSASS!

7. **Find the event where:**
   - SourceImage = powershell.exe
   - TargetImage = lsass.exe

- GrantedAccess shows memory read permissions (0x1010, 0x1038, 0x1410, or 0x1438)

Step 4: Record Key Details

Document these fields from the suspicious event:

- **TimeCreated:** _____
- **SourceImage (full path):** _____
- **TargetImage (full path):** _____
- **GrantedAccess:** _____
- **EventRecordID:** _____

Questions:

1. What access rights (GrantedAccess) were requested? What do they allow?
2. Why is PowerShell accessing LSASS unusual and dangerous?

Phase 2: Verify Privilege Escalation (10 minutes)

Step 5: Open Security_Events_Parsed.csv

1. Open Security_Events_Parsed.csv in Excel
2. Enable **AutoFilter**
3. **Filter EventID column** to show: **4672**
    - This shows "Special Privileges Assigned to New Logon"

Step 6: Find SeDebugPrivilege Usage

4. Look at the **Privileges** column (newly parsed!)
5. **Filter or search** for: **SeDebugPrivilege**
6. You should find **10 events** with this privilege

Step 7: Find the Suspicious One

7. Cross-reference the **TimeCreated** timestamp with your Sysmon Event 10
8. **Look for Event 4672 within 1-2 seconds** of the LSASS access
9. Check the **SubjectUserName** - this shows which user account was used

Questions:

1. Which user account (SubjectUserName) was granted SeDebugPrivilege?
2. Is this the same time as the LSASS memory access?
3. Why is SeDebugPrivilege dangerous? (Hint: It allows reading any process memory, bypassing security)

Phase 3: Investigate PowerShell Activity (10 minutes)

Step 8: Open PowerShell_Events.csv

1. Open PowerShell_Events.csv in Excel
2. **Warning:** This file has 4,644 events - don't try to read everything!
3. Enable **AutoFilter**
4. **Filter EventID column** to show: **4104**
    - This is PowerShell Script Block Logging

Step 9: Search for Attack Indicators

5. Use **Excel Find** (Ctrl+F or Cmd+F):
    - Click "Options" → Search: "Workbook" or "Sheet"
    - Search within: "Values" or "Formulas"
6. **Search the Message column for these keywords (one at a time):**

Keyword	What It Indicates
Invoke-Mimikatz	Mimikatz PowerShell module execution
sekurlsa	Mimikatz credential dumping command
logonpasswords	Mimikatz command to extract passwords
IEX or Invoke-Expression	Dynamic code execution
DownloadString	Downloading code from remote server
-enc or -EncodedCommand	Obfuscated/encoded PowerShell
Empire	PowerShell Empire C2 framework

Step 10: Analyze Malicious Commands

7. When you find suspicious events:
    - **Record the TimeCreated timestamp**
    - **Copy the first 500 characters** of the Message column
    - **Look for command patterns:**
        - Base64-encoded stagers: -enc <long_base64_string>
        - Remote downloads: (New-Object Net.WebClient).DownloadString(...)
        - Mimikatz invocation: Invoke-Mimikatz -Command "sekurlsa::logonpasswords"

Questions:

1. Did you find evidence of Invoke-Mimikatz execution? What timestamp?
2. Were commands base64-encoded? (This is obfuscation to evade detection)

3. Can you identify any C2 (Command & Control) indicators like IP addresses or domains?

Phase 4: Build Attack Timeline (5 minutes)

Step 11: Correlate All Evidence

Using your findings from all three log sources, build a timeline:

Time (UTC)	Log Source	Event ID	What Happened	Significance
14:XX:XX	PowerShell	4104	PowerShell executed with -enc parameter	Initial Empire stager execution
14:XX:XX	PowerShell	4104	Found Invoke-Mimikatz in Message	Attacker loads Mimikatz module
14:XX:XX	Security	4672	SeDebugPrivilege granted to pgustavo	Privilege escalation for memory access
14:XX:XX	Sysmon	10	powershell.exe → lsass.exe (0x1410)	**LSASS memory dumped**
14:XX:XX	PowerShell	4104	sekurlsa::logonpasswords command	Credential extraction

Fill in the actual timestamps from your investigation!

Exercise Questions
Detection Questions:

1. **What was the first indicator of compromise?**
   - Network connection to external IP
   - PowerShell process accessing LSASS
   - File creation in temp directory
   - User logon from unusual location
2. **Which Windows privilege was abused to access LSASS memory?**
   - Answer: _____
3. **What tool/technique was used to dump credentials?**
   - Answer: _____

Analysis Questions:

4. **Why is this attack particularly dangerous?**
   - Hint: Consider the target system role (Domain Controller)
   - Answer: _____
5. **List at least 3 Indicators of Compromise (IOCs):**
   - Process: _____
   - Command: _____

- Privilege: _____

6. **What MITRE ATT&CK technique was used?**
   - Answer: T1003.001 - OS Credential Dumping: LSASS Memory

Response Questions:

7. **What immediate containment actions should you take?**
   - Isolate the compromised domain controller
   - Disable the pgustavo user account
   - Force password reset for all domain users
   - Review authentication logs for lateral movement
   - Check for other systems accessed by pgustavo account
8. **What evidence would you preserve for further investigation?**
   - Answer: _____

Tips for Success

Excel Filtering Techniques:

- **Multiple filters:** Use Filter dropdown → check/uncheck values
- **Text search:** Filter → Text Filters → Contains
- **Date/time filtering:** Filter → Date Filters → Custom
- **Copy filtered results:** Select visible cells → Ctrl+C → Paste to new sheet

Keyword Searching:

- Use **Ctrl+F (Windows)** or **Cmd+F (Mac)** for Find
- Search in: **"Workbook"** to search entire file
- **Find All** shows all matches with locations
- **Case-insensitive** searching works for most keywords

Common Pitfalls:

- Reading every event sequentially (too slow!)
- Ignoring timestamps (can't build timeline)
- Not correlating across log sources (incomplete picture)
- **Do this instead:** Start with highest-fidelity alert (Sysmon Event 10), then work backward and forward

Time Management:

- Minutes 0-15: Find LSASS access (Sysmon)
- Minutes 15-25: Verify privileges (Security)
- Minutes 25-35: Investigate PowerShell
- Minutes 35-40: Build timeline

Module 4: Detection Rule Development

Detection rules turn defensive knowledge into repeatable monitoring. Instead of relying on memory or intuition alone, analysts and detection engineers create structured logic that can identify suspicious behavior consistently across many systems.

This module introduces detection engineering at an entry level. Students learn that a good rule begins with a clear idea of the behavior being detected, the data source required, and the fields that make the behavior distinguishable from normal activity.

Students will work with Sigma because it provides a platform-neutral way to describe detections. That makes it easier to understand the detection concept before translating it into a specific technology such as Splunk. This is valuable for students because it teaches the logic behind the rule, not just the syntax of one tool.

A strong introductory rule should answer several questions: What does this detect? Why does it matter? What data is required? What legitimate activity might trigger it? How would we tune it in a real environment? These are the same questions professional detection engineers ask every day.

This module also emphasizes the tradeoff between false positives and false negatives. If a rule is too broad, analysts waste time. If a rule is too narrow, the attacker is missed. Effective blue team work requires balancing both while documenting assumptions and tuning ideas.

By the end of this module, students should be able to write simple detection content, validate it against data, and explain how that content would support a real SOC.

Exercise 4: Write Detection Rules

Objectives

Scenario: You are a junior SOC analyst who has been asked to build detection content for common attacker behaviors. Your task is to write five beginner-friendly Sigma rules and translate each one into a basic Splunk search that could be used to hunt for or alert on the same activity.

Goal: By the end of this exercise, you will produce a small detection library that covers five high-value attack techniques and explain why each rule is useful.

Attacks You Must Detect

- PowerShell encoded command execution — MITRE ATT&CK T1059.001. Look for PowerShell launched with encoded or hidden execution options.
- Lateral movement via PsExec — MITRE ATT&CK T1021.002. Look for PsExec service creation or remote command execution patterns.
- Persistence via Registry Run key — MITRE ATT&CK T1547.001. Look for programs being added to Run or RunOnce registry keys.
- Suspicious scheduled task creation — MITRE ATT&CK T1053.005. Look for scheduled tasks created to launch scripts or suspicious binaries.
- Credential access via LSASS dump — MITRE ATT&CK T1003.001. Look for tools or commands attempting to dump LSASS memory or access credentials.

Expected Student Deliverables

- Five Sigma rules written in YAML format.
- Five Splunk searches that match the same detection logic.
- A short note for each rule describing what it detects.
- A short note for each rule listing at least one likely false positive or tuning consideration.

Environment and Assumptions

- Students may write rules in any text editor such as Notepad, VS Code, or Word if needed.
- Students should assume Windows log sources such as Security logs, Sysmon, PowerShell logs, and endpoint telemetry are available.

- Splunk syntax in this exercise is introductory and focuses on clear logic instead of production-level optimization.
- If sample logs are provided by the instructor, students should test whether their searches would identify the expected activity.

Detection Rule Writing Template

Use the following simple structure for each rule. You do not need to memorize every Sigma field; focus on the fields below.

Field	What To Put Here
title	A short rule name such as Suspicious PowerShell Encoded Command
id	A unique identifier if your instructor wants one
status	Use values such as experimental or test
description	One or two sentences describing the suspicious behavior
logsource	What platform and log source the rule depends on, such as product: windows and category: process_creation
detection	The actual matching logic, such as process name, command-line arguments, or registry path
condition	Usually the name of the selection, such as selection
fields	Optional fields you want returned, such as Image, CommandLine, User, ParentImage, ComputerName

Step 1 – Setup Work Environment

Open a blank document or text editor where you can save your five rules. Create five clearly labeled sections: Rule 1 through Rule 5.

Step 2 – First Attack Requirement

Read the first attack requirement: PowerShell encoded command execution. Ask yourself two simple questions: What process would I expect to see? What command-line text would make it suspicious?

Step 3 – Begin Rule 1

For Rule 1, start by writing the title, description, and MITRE technique. Then set the log source to Windows process creation data. In the detection section, look for powershell.exe or pwsh.exe combined with terms such as -enc, -encodedcommand, or hidden window execution flags.

Step 4 – Begin Rule 2

Repeat the same process for Rule 2, which covers lateral movement via PsExec. Think about what artifacts PsExec commonly creates, such as psexec.exe, service creation events, or a service name like PSEXESVC.

Step 5 – Rule 3

Repeat the process again for Rule 3, persistence via Registry Run key. Focus on registry modification events or endpoint telemetry that records changes to Run and RunOnce keys. Your detection should look for writes to those paths.

Step 6 – Rule 4

For Rule 4, suspicious scheduled task creation, look for command lines involving schtasks.exe /create, task registration events, or tasks launching scripts from unusual directories such as AppData, Temp, Public, or user profile locations.

Step 7 – Rule 5

For Rule 5, credential access via LSASS dump, think about what a defender might catch: procdump targeting lsass.exe, rundll32 comsvcs.dll MiniDump usage, suspicious handle access to lsass.exe, or known dumping command patterns. Write the Sigma rule.

Step 8 - Rule Check

When all five rules are drafted, go back through them one at a time and make sure each rule includes: a title, a description, a log source, detection logic, a condition, at least one false positive, a severity level, and a MITRE ATT&CK mapping.

Step 9 – Document Rules

Finally, add a one- or two-sentence analyst note under each rule answering: Why would this activity matter to a SOC analyst, and what should the analyst check next?

Suggested Starter Logic for Each Rule

Rule 1: PowerShell Encoded Command

- Possible data fields: Image, CommandLine, ParentImage, User, ComputerName
- Starter indicators: powershell.exe, pwsh.exe, -enc, -encodedcommand, FromBase64String, -nop, -w hidden
- Analyst follow-up: Determine who launched the command, from what parent process, and whether the script contacted an external IP or created a file.

Rule 2: PsExec Lateral Movement

- Possible data fields: Image, CommandLine, ServiceName, TargetHost, User
- Starter indicators: psexec.exe, PSEXESVC, service creation on remote systems, ADMIN$ usage
- Analyst follow-up: Identify the source host, destination host, account used, and whether the action was administrative or malicious.

Rule 3: Registry Run Key Persistence

- Possible data fields: TargetObject, Details, Image, User, ComputerName
- Starter indicators: \Software\Microsoft\Windows\CurrentVersion\Run, RunOnce, writes to suspicious executable or script paths
- Analyst follow-up: Confirm whether the autorun target is legitimate software or a malicious payload.

Rule 4: Scheduled Task Creation

- Possible data fields: Image, CommandLine, TaskName, User, ComputerName
- Starter indicators: schtasks.exe /create, task registration events, suspicious tasks pointing to PowerShell, cmd.exe, wscript.exe, or files in Temp/AppData
- Analyst follow-up: Determine whether the task was created by IT administration, software installation, or an attacker establishing persistence.

Rule 5: LSASS Dumping

- Possible data fields: Image, CommandLine, TargetImage, GrantedAccess, ParentImage, User
- Starter indicators: procdump.exe -ma lsass.exe, comsvcs.dll MiniDump, suspicious access to lsass.exe, process access telemetry involving LSASS
- Analyst follow-up: Check whether credential dumping succeeded, whether tickets or hashes were later used, and whether the host should be isolated.

Module 5: SIEM Query Mastery

A SIEM brings security data together so analysts can search, correlate, and investigate events quickly. For many blue team roles, the SIEM is the primary workspace for daily operations. It is where alerts are reviewed, timelines are built, and suspicious activity is confirmed or dismissed.

This module introduces the basic workflow of working inside a SIEM: confirm data ingestion, understand the available fields, start with a broad search, then narrow the scope using filtering, statistics, and pivots. Students should think of queries as investigation tools, not just commands to memorize.

Strong SIEM use depends on asking focused questions. Which source IP generated repeated failures? Which system is sending the most outbound data? Which destination appears on a fixed interval and may represent beaconing? Good analysts move from broad observations to targeted validation.

This section also reinforces the importance of data quality. A query is only as good as the underlying fields and parsing. Students should pay attention to indexes, source types, field names, and event counts so they can trust their results and troubleshoot when something does not look right.

The goal of this module is not just to teach syntax. It is to help students build confidence using a SIEM to answer real investigation questions under time pressure.

By the end of this module, students should be able to load data, validate it, run focused searches, and use query results to explain what happened in a practical SOC scenario.

Exercise 5: SIEM Investigation Scenarios

Objective

A data file contains log records spanning a simulated 8-hour SOC shift (2025-02-20 08:00–16:00). Three distinct attack scenarios are embedded within realistic background noise.

- Import data into Splunk from three different sources that are in a single CSV file.
- Analyze the data to discover any potential issues that may have occurred.

Provided Dataset

- **Ex5_ThreatHunting_Dataset.csv**

Step 1 – Log Into Splunk

From the Windows 11 desktop, open a browser and browse to:

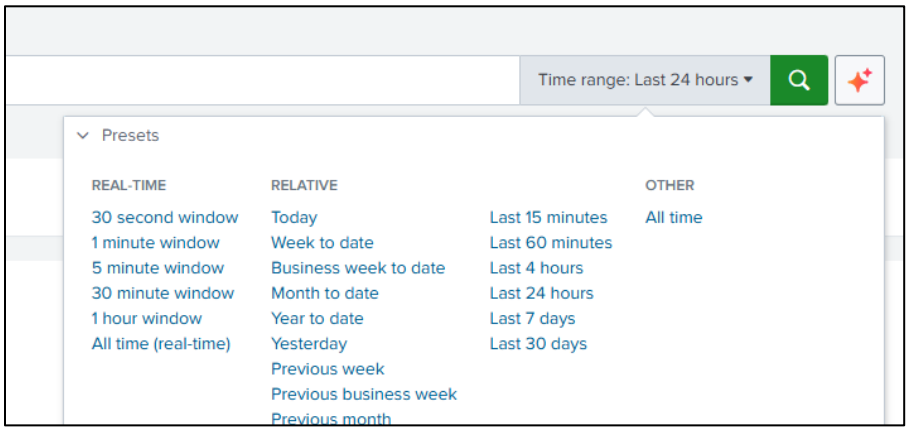
http://10.1.1.201

Login using the username and password (BlueTeam2026!) given to you by the instructor.

Step 2. Validate Data Access

Run these quick SPL queries in **Splunk Search** to confirm the data is accessible.

NOTE: Make sure the Time range is set to All Time.



1. Get total event count:

```
index=ex5_siem | stats count
```

2. Get events per log source:

```
index=ex5_siem | stats count by log_source
```

3. Types of action:

```
index=ex5_siem | stats count by action | sort -count
```

4. See if the MITRE techniques are present:

```
index=ex5_siem mitre_tech!="" | stats count by mitre_tech
```

Step 3. Threat Hunting

For the purposes of this exercise, there are three conditions under investigation: a brute force attack (ATT&CK Technique T1110); data exfiltration (ATT&CK Technique T1041); and C2 malware beaconing (ATT&CK Technique T1071.001).

All of the following queries take place from the **Splunk Search** screen.

Brute Force Attack
Identify failed login attempts:

```
index=ex5_siem action=failed |   stats count by src_ip, user |
where count > 10 | sort -count
```

Expected: src_ip 203.0.113.50 with 35 failed attempts against user admin. Multiple 198.51.100.x IPs represent a distributed attack.

Data Exfiltration
Identify large amounts of data being exfiltrated:

```
index=ex5_siem log_source=firewall01 action=allowed |
stats sum(bytes_out) as total_bytes_out by dest_ip |
eval MB=round(total_bytes_out/1024/1024, 2) |
where MB > 10 | sort -MB
```


Expected: 185.220.101.45 shows 10–30 MB of outbound data — a clear exfil signal.

C2 Beaconing
Identify end points "phoning home":

```
index=ex5_siem log_source=proxy01 | bin _time span=5m
| stats count by _time, dest_ip | where count >= 4 | sort _time
dest_ip
```

Expected: 192.0.2.200 appears every 5-minute bucket with ~5 connections — classic beacon pattern.

The Full Attack
Identify the entire attack:

```
index=ex5_siem mitre_tech!=""
| table _time, log_source, src_ip, dest_ip, user, action,
bytes_out, mitre_tech
| sort _time
```

Expected: A clear progression — Brute Force → Valid Account success → Exfil begins.

Step 4. ATT&CK Navigator Mapping

After completing queries, document findings in ATT&CK Navigator:

- Go to https://mitre-attack.github.io/attack-navigator/
- Click **Create New Layer** → Enterprise.
- Use the search bar to find and highlight each technique:
  - T1110 (Brute Force) - set color Red
  - T1078 (Valid Accounts) - set color Orange
  - T1041 (Exfil over C2) - set color Orange
  - T1071.001 (Web Protocols / Beaconing) - set color Yellow
- Export the layer: Layer → Download as PNG or JSON.

Module 6: Living Off the Land Detection

Many modern attackers prefer to abuse legitimate operating system tools instead of dropping obvious malware. This tactic is often called Living Off the Land. It is effective because the binaries are trusted, already installed, and commonly allowed by security controls.

For blue team defenders, that means process names alone are not enough. Tools such as certutil, bitsadmin, mshta, and regsvr32 may be used legitimately, but the command-line arguments, parent process, timing, destination, and user context often reveal when the behavior is suspicious.

This module teaches students to focus on behavior over reputation. A signed Microsoft binary is not automatically safe. Analysts must ask what the tool is doing, where it is connecting, what file path it is touching, and whether that pattern makes sense for the host and user.

Students will also see why command-line visibility and network context matter. A binary launching with download flags, writing to unusual folders, or contacting rare external IPs may indicate staging, persistence, or command-and-control activity even when no traditional malware alert exists.

This topic is valuable because it reflects real attacker tradecraft and teaches students how to investigate subtle abuse that might bypass simple signature-based defenses.

By the end of this module, students should be able to recognize suspicious LOLBin patterns and explain why behavioral detection is essential in modern blue team operations.

Exercise 6: Living-off-the-Land Detection

Objective

The provided dataset simulates Sysmon-style process and network telemetry collected across a corporate Windows environment over approximately 4 hours on 2026-03-09. It contains 145 log rows across 10 hosts and 10 users, with 4 hidden attack scenarios embedded in legitimate noise.

- Import data into Splunk from three different sources that are in a single CSV file.
- Analyze the data to discover any potential issues that may have occurred.

Provided Dataset

- **Ex6_LOLBin_Dataset.csv**

Step 1 – Log Into Splunk

From the Windows 11 desktop, open a browser and browse to:

http://10.1.1.201

Username: Assigned by instructor
Password: BlueTeam2026!

Splunk Analysis Queries -- The Hunt

Work through these SPL queries in order. Each one narrows the focus from broad discovery down to precise attack confirmation and attacker-pivot analysis. In a real environment, these are typical hunt patters that would be performed by your SOC analysts.

Step 1 – Broad LOLBin Activity Overview

Start wide -- surface all LOLBin process events by host and user to find high-volume anomalies.

LOLBin Process Count by Host/User:

```
index=ex6_lolbin event_type="ProcessCreate"
 (process_name="certutil.exe"  OR process_name="bitsadmin.exe"
  OR process_name="mshta.exe"  OR process_name="regsvr32.exe")
```

```
| stats count BY extracted_host, user, process_name
| sort  - count
```

What to Look For:

- Any single user running the same LOLBin many times (volume anomaly).
- LOLBins spawned under unusual parent processes (powershell.exe, explorer.exe).
- Hosts that have never normally used these tools.

Document your findings.

Step 2 - certutil Remote Download Detection (T1105 / T1218.001)

Filter certutil command lines for the specific flags used in remote downloads and base64 decode operations.

certutil Malicious Flag Detection:

```
index=ex6_lolbin process_name="certutil.exe"
| eval is_malicious=if(
    match(command_line, "(?i)-urlcache") OR
    match(command_line, "(?i)-split")    OR
    match(command_line, "(?i)http[s]?://") OR
    match(command_line, "(?i)-decode"),
    "SUSPICIOUS", "Likely Legitimate")
| table timestamp, extracted_host, user, parent_process,
command_line, is_malicious
| sort  timestamp
```

Expected Findings

- 7 events flagged SUSPICIOUS on WKSTN-07 for user jsmith.
- Payloads staged in C:\Windows\Temp\ and C:\Users\jsmith\AppData\Roaming\.
- External IPs: 185.220.101.45 and 203.0.113.77 -- flag for threat intel lookup.
- Legitimate certutil usage (hash checks, store queries) shows "Likely Legitimate".

Document your findings.

Step 3 - bitsadmin Malicious Transfer Detection (T1197)

Detect bitsadmin used for file download transfers vs. legitimate list/monitoring operations.

bitsadmin Transfer and Persistence Detection

```
index=ex6_lolbin process_name="bitsadmin.exe"
| eval is_malicious=if(
    match(command_line, "(?i)/transfer") OR
    match(command_line, "(?i)/download") OR
    match(command_line, "(?i)SetNotifyCmdLine"),
    "SUSPICIOUS", "Likely Legitimate")
| table timestamp, extracted_host, user, parent_process,
command_line, dest_ip, is_malicious
| sort  timestamp
```

Key Indicator – SetNotifyCmdLine:

- The /SetNotifyCmdLine flag tells BITS to execute a program when a job completes.
- Attackers use this as a persistence mechanism so malware re-runs automatically.
- Any SetNotifyCmdLine pointing to Temp directories is highly suspicious.

Step 4 - mshta Remote HTA Execution (T1218.005)

Detect mshta executing remote URLs or inline VBScript/JavaScript -- a common phishing and macro-bypass technique.

mshta Inline Script / Remote Payload Execution:

```
index=ex6_lolbin process_name="mshta.exe"
| eval suspicious_pattern=case(
    match(command_line, "(?i)http[s]?://"),   "Remote HTA URL",
    match(command_line, "(?i)vbscript:"),      "Inline VBScript",
    match(command_line, "(?i)javascript:"),    "Inline
JavaScript",
    1=1,                                       "Review Manually")
| table timestamp, extracted_host, user, parent_process,
command_line, suspicious_pattern
| sort  timestamp
```

Step 5 - Squiblydoo -- regsvr32 Remote SCT (T1218.010)

"Squiblydoo" is a classic AppLocker bypass: regsvr32 with /i:http:// loads and runs remote COM scriptlets without touching disk.

Squiblydoo regsvr32 AppLocker Bypass:

```
index=ex6_lolbin process_name="regsvr32.exe"
```

```
| eval squiblydoo=if(
    match(command_line, "(?i)/i:http[s]?://") AND
    match(command_line, "(?i)scrobj\.dll"),
    "SQUIBLYDOO DETECTED", "Review")
| table timestamp, extracted_host, user, parent_process,
command_line, squiblydoo
| sort  timestamp
```

Step 6 - Network Connections FROM LOLBins (Sysmon EID 3)

Correlate Event ID 3 (NetworkConnect) events to confirm which LOLBins made outbound connections.

Outbound Network Activity from LOLBin Processes

```
index=ex6_lolbin event_id=3
  (process_name="certutil.exe"  OR process_name="bitsadmin.exe"
   OR process_name="mshta.exe"  OR process_name="regsvr32.exe"
   OR process_name="svchost.exe")
| table timestamp, extracted_host, user, process_name, dest_ip,
dest_port, bytes_out
| sort  - bytes_out
```

Note on svchost.exe in BITS Results:

- BITS transfers execute under svchost.exe -- not bitsadmin.exe -- for the actual download.
- Use dest_ip values from Hunt 3 to confirm svchost connections belong to the BITS job.

Step 7 - Attacker Pivot -- Full Timeline for jsmith / WKSTN-07

Students who identify jsmith should pivot to see ALL activity on WKSTN-07 -- revealing the full multi-stage attack chain.

Full Attack Timeline: jsmith on WKSTN-07:

```
index=ex6_lolbin extracted_host="WKSTN-07" user="jsmith"
| table timestamp, event_type, parent_process, process_name,
command_line, dest_ip, bytes_out
| sort  timestamp
```


Attack Chain Discovery -- WKSTN-07 / jsmith

- 06:14-06:17 certutil downloads payload (update.exe, stage2.ps1, encoded.txt decode)
- 07:47 certutil downloads additional tools (tools.zip, svchost32.exe)
- 07:58 certutil: C2 check-in via -verifyctl to 203.0.113.77
- 07:58-08:04 regsvr32 Squiblydoo fires -- loads remote SCT from 192.0.2.55 and 203.0.113.77
- IPs 185.220.101.45 and 192.0.2.55 appear across both stages -- confirmed campaign infrastructure.

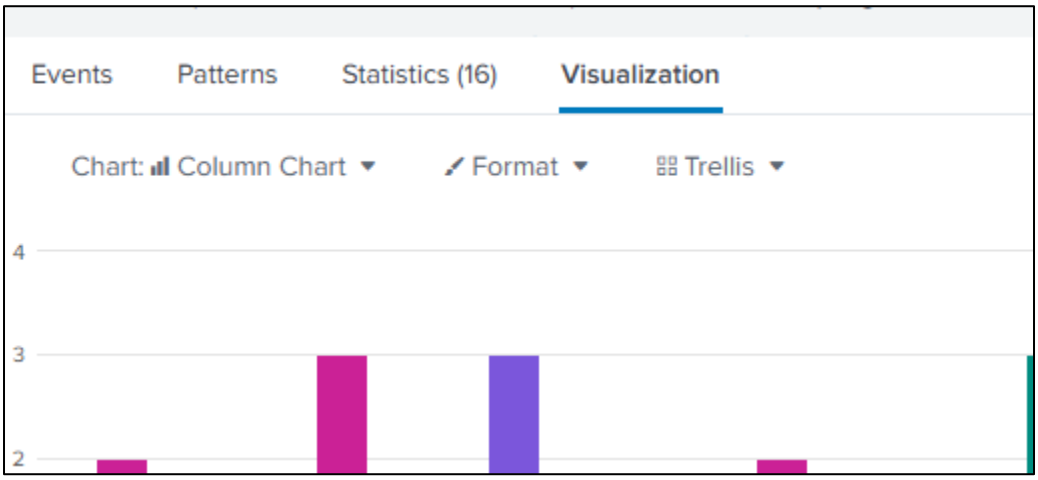
Step 8 - Attack Timeline Visualization

Use timechart to visualize when each LOLBin was active. Spikes indicate coordinated or automated attack phases.

LOLBin Activity Timechart (15-minute buckets)

```
index=ex6_lolbin event_type="ProcessCreate"
 (process_name="certutil.exe"  OR process_name="bitsadmin.exe"
  OR process_name="mshta.exe"  OR process_name="regsvr32.exe")
| timechart span=15m count BY process_name
```

Click on the Visualization tab to see the timeline.



Step 9 - MITRE ATT&CK Navigator Mapping

Map your findings at https://attack.mitre.org/navigator/ to complete your exercise documentation.

Technique ID	Technique Name	Sub-Technique	Scenario	Severity
T1105	Ingress Tool Transfer	N/A	certutil download	High
T1218.001	Sys Binary Proxy - CertUtil	T1218	certutil decode	High
T1197	BITS Jobs	N/A	bitsadmin transfer	High
T1218.005	Sys Binary Proxy - Mshta	T1218	mshta HTA execution	High
T1218.010	Sys Binary Proxy - Regsvr32	T1218	Squiblydoo SCT load	Critical
T1036.005	Masquerading - Match Legit	T1036	svchost32.exe naming	Medium

Module 7: Zero Trust Architecture

Zero Trust is a defensive design approach built on a simple idea: no user, device, workload, or network location should be trusted automatically. Every access request should be evaluated based on identity, device health, privilege, and policy.

For blue team professionals, Zero Trust is important because it reduces the damage an attacker can do after the initial compromise. If identity is strongly verified, access is limited, and networks are segmented, then phishing, credential abuse, and lateral movement become harder to execute at scale.

This module introduces Zero Trust at a practical level. Students do not need to build a full enterprise architecture. Instead, they should learn to identify critical assets, define trust boundaries, apply least privilege, require stronger identity controls, and place monitoring where it will reveal misuse.

Students should also understand that Zero Trust is not one product. It is a design philosophy supported by multiple controls such as MFA, conditional access, network segmentation, endpoint health checks, privileged access management, and centralized logging.

A strong beginner understanding of Zero Trust helps students think beyond incident response alone. It helps them design environments that are easier to defend and harder for attackers to traverse.

By the end of this module, students should be able to propose practical Zero Trust controls that limit unauthorized access and reduce the blast radius of compromise.

ATT&CK Technique / Mitigation	ID	Relevance to This Exercise
Valid Accounts	T1078	Prevent credential misuse with MFA + conditional access
Lateral Movement (SMB/RDP)	T1021	Micro-segmentation blocks east–west pivoting
Use Alternate Auth Material	T1550	Passwordless + device compliance mitigates token theft
Persistence via Svc Install	T1543	JIT access & change-monitoring detect rogue services
Data from Cloud Storage	T1530	CSPM + least-privilege storage policies limit exposure
Remote Services	T1021.001	Bastion hosts + JIT RDP remove standing access

M1032 – Multi-factor Auth	**Mitigation**	Core ZT control across all asset tiers
M1030 – Network Segmentation	**Mitigation**	Micro-segmentation per asset in the design
M1018 – User Account Mgmt	**Mitigation**	Least-privilege + JIT privileged access management

Exercise 7 – Zero Trust Design Challenge

Objective

Students apply Zero Trust principles to a small hybrid environment and design controls that reduce unauthorized access, lateral movement, and attacker persistence.

Scenario

Use the following scenario as your design context throughout the exercise:

Asset #	Description
Asset 1	On-premises Windows File Server (contains HR records; high sensitivity)
Asset 2	Azure SQL Database (customer PII; PCI-DSS in scope)
Asset 3	Internal Web Application Server (intranet portal; medium sensitivity)
Asset 4	On-premises Domain Controller (AD authentication hub; critical)
Asset 5	Azure Blob Storage (marketing content + backups; mixed sensitivity)

Hybrid environment details:

- 250 employees,
- 3 remote offices,
- Azure AD (Entra ID) used for identity,
- Microsoft Defender for Endpoint on all workstations,
- Palo Alto firewall on-premises,
- Azure NSGs and VNet peering for cloud resources.
- No current ZT controls are in place — your task is to design and document them.

Step 1 – Review Zero Trust Fundamentals

Read the NIST SP 800-207 Executive Summary (nvlpubs.nist.gov/nistpubs/SpecialPublications/NIST.SP.800-207.pdf) or recall the three pillars. Pay particular attention to the concept of 'policy enforcement points' (PEP) and 'policy decision points' (PDP) - these will form the core of your diagram. Optionally, review the CISA Zero Trust Maturity Model (cisa.gov/zero-trust-maturity-model) to understand maturity levels (Traditional → Initial → Advanced → Optimal).

Once you are comfortable with these concepts, move on to Step 2.

Step 2 - Classify the Five Assets

For each asset in the scenario table, assign a sensitivity tier (High / Medium / Low) and identify the primary risk if compromised (e.g., data exfiltration, ransomware pivot, auth bypass). Record this in a simple 3-column table in your notes.

Step 3 - Define Trust Boundaries and Access Paths

Map where users, devices, and applications connect from. Identify trust zones such as user endpoints, administrative workstations, server networks, DMZ resources, and cloud services.

Step 4 – Design Network Segmentation

Propose segmentation to isolate each asset.
On premises: assign dedicated VLANs and inter-VLAN firewall ACLs (deny all, permit only required ports/protocols).

Cloud: assign each Azure asset a separate subnet within a VNet; apply NSGs with minimal allow rules.

Define a DMZ subnet for the internal web application server. Prevent direct workstation-to-server east–west traffic (this blocks T1021 lateral movement).

Step 5 – Define Identity Controls

For each asset, specify identity enforcement:

- Require MFA for all users via Entra ID Conditional Access — enforce for all sign-ins regardless of network location.
- Configure Conditional Access policies: block sign-in from non-compliant devices and from high-risk sign-in locations.
- Implement Privileged Identity Management (PIM) for Domain Controller and Azure SQL admin access — require approval + justification + time-limited activation (JIT).
- Apply least-privilege RBAC: no standing Global Admin; File Server read/write roles scoped per department.

Step 6 - Define Device and Workload Trust Requirements

Specify what conditions a device or workload must meet before access is allowed. Examples include EDR health, encryption enabled, approved operating system, compliant device state, or workload identity validation.

Step 7 – Specify Continuous Monitoring

Identify data sources and monitoring controls for each asset. At minimum, specify:

- Entra ID sign-in logs (for anomalous auth),
- Azure Activity Log (for control-plane changes),
- Microsoft Defender for Endpoint alerts (for malware/LOLBin on workstations),
- Windows Security Event Log (4624/4625/4688 for logon and process events),
- Azure Monitor / Sentinel (SIEM aggregation and alerting).

Define at least three alert rules — e.g., 'Admin sign-in outside business hours', 'Mass file access on File Server', 'SQL export > 100 MB'.

Step 8: Map Controls to MITRE ATT&CK

Use ATT&CK Navigator to connect your design to techniques such as Valid Accounts (T1078), Remote Services (T1021), Use Alternate Authentication Material (T1550), and persistence-related behaviors. Note which controls reduce or detect each technique.

Step 9 – Build the ATT&CK Navigator Layer

Open attack.mitre.org/navigator. Create a new layer named 'ZT Design – Exercise 7'. Navigate to the Enterprise matrix. Tag the following techniques in your layer, using green for mitigated and amber for partially mitigated:

Step 10: Write the Design Rationale

Prepare a short summary explaining why your architecture is effective, where residual risks remain, and what trade-offs exist between usability, performance, cost, and security strength.

Module 8: Alert Triage Simulation

SOC analysts spend much of their time triaging alerts. That means deciding what matters first, what needs more evidence, what can be closed safely, and what should be escalated immediately. Triage is where technical knowledge meets decision-making under time pressure.

This module introduces a simple triage mindset: assess severity, understand asset criticality, look at user and host context, and determine whether the alert is isolated or part of a larger pattern. Analysts do not need complete certainty before acting, but they do need a defensible reasoning process.

Students should learn that not every alert deserves the same response. Some are false positives, some are low-risk noise, and some represent the early stages of a serious incident. The challenge is to separate those categories quickly without missing the signals that connect multiple alerts into one attack story.

This section also emphasizes documentation. Good triage notes explain what was reviewed, what evidence was found, what decision was made, and what should happen next. Clear documentation allows another analyst or an incident responder to continue the work without starting over.

The goal is to help students build confidence making practical decisions in a queue-based environment that resembles real SOC work.

By the end of this module, students should be able to prioritize alerts, pivot on key fields, recognize related activity, and document their conclusions clearly.

Exercise 8 – Real-World Alert Queue

Objectives

- Prioritize alerts using severity, context, and affected asset criticality.
- Separate false positives from actionable threats.
- Correlate related alerts into a single attack narrative.
- Document triage decisions and escalation notes clearly.
- Map the discovered campaign to MITRE ATT&CK tactics and techniques.

Provided Dataset

- **Ex8_Real_World_Alert_queue.csv**

Step 1 – Log Into Splunk

From the Windows 11 desktop, open a browser and browse to:

http://10.1.1.201

Username: Assigned by instructor
Password: BlueTeam2026!

Step 2 - Display the alert queue in a readable format.

```
index=ex8_alerts | table alert_id timestamp severity host
username src_ip dest_ip alert_type description queue_status |
sort timestamp
```

Step 3 – Identify HIGH Alerts

Read down the table and write down every High severity alert ID. You should see A-007, A-008, A-014, and A-018.

Step 4 – Look for Anomalies

Look for repeated values in the username field. User mturner appears multiple times and is a strong pivot candidate.

Step 5 – Dig Deeper

Run the following username pivot to view all alerts for mturner:

```
index=ex8_alerts username="mturner" | table timestamp alert_id
severity extracted_host username src_ip dest_ip alert_type
description | sort timestamp
```

Step 6 – Update Your Documentation

Review the results and note that A-006, A-007, and A-008 all involve user mturner.

Step 7 – Investigate Further

Now pivot on the host field to verify the same pattern on WKSTN-14:

```
index=ex8_alerts extracted_host="WKSTN-14" | table timestamp
alert_id severity host username src_ip dest_ip alert_type
description | sort timestamp
```

Notice that A-007 and A-008 also share the destination IP 203.0.113.77. Run a final pivot on that destination IP:

```
index=ex8_alerts dest_ip="203.0.113.77" | table timestamp
alert_id severity extracted_host username src_ip dest_ip
alert_type description | sort timestamp
```

Step 8 – Critical Thinking

Decide whether the three mturner alerts represent one attack chain. A reasonable conclusion is: failed VPN login burst against mturner, then PowerShell download on WKSTN-14, then beaconing to the same external IP.

Step 9 – Complete the Analysis

Classify the remaining alerts as likely benign, needs monitoring, or escalated. For example, A-002 and A-020 look consistent with backups, while A-018 may still deserve review but does not link to the mturner campaign.

Choose the single alert you would escalate first and explain why. Many students will choose A-008 because active beaconing suggests ongoing command-and-control.

Module 9: Ransomware Kill Chain Analysis

Ransomware incidents are rarely just encryption events. In many modern cases, the attacker first gains access, establishes execution, steals credentials, moves laterally, stages data for exfiltration, and only then deploys the final impact phase. Understanding that progression is critical for defense.

This module helps students break ransomware into phases they can recognize and investigate. Instead of seeing ransomware as a single outcome, they learn to identify earlier opportunities for detection such as phishing, PowerShell abuse, unusual authentication, archive creation, or large outbound transfers.

Students should pay attention to how ransomware operations often combine techniques. The same intrusion may involve user execution, credential abuse, persistence, data theft, and destructive impact. That makes timeline building especially important because it shows both the sequence of the attack and where defenders could have intervened.

This section also reinforces the business impact of blue team decisions. When a file server or domain-connected system is involved, the incident quickly becomes broader than a single host. Analysts must consider containment, scope, and recovery, not just the initial malicious event.

The lesson is that earlier detection matters. If defenders identify the attack during execution or lateral movement, they may prevent both exfiltration and encryption.

By the end of this module, students should be able to reconstruct a ransomware story, identify the major phases, and explain where defensive controls could have reduced the impact.

Exercise 9 – Ransomware Investigation

Objective

Students reconstruct a simplified ransomware timeline and explain how the intrusion moved from phishing to data theft and encryption.

Provided Dataset

- **Ex9_LockBit_Investigation_Timeline.csv**

Step 1 – Log Into Splunk

From the Windows 11 desktop, open a browser and browse to:

http://10.1.1.201

Username: Assigned by Instructor
Password: BlueTeam2026!

Step 2 – Confirm Data Has Loaded

Run the following search:

```
index=ex9_ransomware | stats count
```

The record count should be 10.

Step 3 – Sort the Data In Chronological Order

```
index=ex9_ransomware | sort timestamp | table timestamp
event_type extracted_host username ip_or_dest process_name
description mitre_technique severity
```

Step 4 – Identify Initial Access Event

Read the first row and identify the likely initial access event. The dataset starts with an email event on WKSTN-22 for user mgarcia

Step 5 - Pivot

Pivot on username mgarcia to follow the compromised user from the workstation to the file server:

```
index=ex9_ransomware username="mgarcia" | sort timestamp | table
timestamp event_type extracted_host username ip_or_dest
process_name description
```

Locate the row that shows access to SRV-FILE-01 and explain why it suggests lateral movement or credential misuse.

Step 6 – Pivot Again

Pivot on host SRV-FILE-01 to focus on the server phase of the incident:

```
index=ex9_ransomware host="SRV-FILE-01" | sort timestamp | table
timestamp event_type extracted_host username ip_or_dest
process_name description severity
```

Identify the row that suggests data exfiltration. In this dataset, the outbound archive upload to 203.0.113.200 is the clearest exfiltration indicator.

Identify the rows that suggest ransomware impact. vssadmin.exe, lockbit_readme.txt, and lockbit.exe are the strongest impact indicators.

Step 7 – Document Your Findings

Write the final attack story in order: phishing email, malicious document execution, PowerShell payload download, file server access, external upload, deletion of shadow copies, ransom note creation, and encryption.

Module 10: Threat Hunting Methodology

Threat hunting is the proactive search for suspicious activity that has not yet produced a clear alert. Instead of waiting for the SIEM to tell them what is wrong, analysts start with a hypothesis and look for patterns that suggest an adversary may already be present.

This module introduces hunting as a structured process. A useful hunt begins with a question such as: Are we seeing suspicious persistence on user workstations? Are rare parent-child process relationships appearing? Is there after-hours activity that suggests staging or command-and-control?

For entry-level students, the most important lesson is that hunting is not random searching. Effective hunting relies on known tradecraft, good scoping, careful pivots, and clear documentation of what was reviewed and why the findings matter.

Students should also understand that hunting often begins with a weak signal rather than a high-confidence alert. A rare scheduled task, an odd registry change, or repeated high-risk events on one host may not prove compromise by themselves, but together they can reveal a meaningful attack pattern.

This section encourages students to combine ATT&CK knowledge, log analysis, and analytical reasoning to move from suspicion to evidence.

By the end of this module, students should be able to state a hunting hypothesis, investigate it with focused queries, and document whether the evidence supports escalation.

Exercise 10 – APT Threat Hunt

Objective

Students perform a focused hunt in a large event dataset to identify a suspicious persistence and command-and-control pattern hidden in routine activity.

Provided Dataset

- **Ex10_APT_Threat_Hunt_Dataset.csv**

Step 1 – Log Into Splunk

From the Windows 11 desktop, open a browser and browse to:

http://10.1.1.201

Username: Assigned by Instructor
Password: BlueTeam2026!

Step 2 – Confirm Data Has Loaded

Run the following search:

```
index=ex10_hunt | stats count
```

The record count should be 2000.

Step 3 – Begin With Event Types

Start broad and run the following query to see how many events exist for each event type:

```
index=ex10_hunt | stats count by event_type | sort - count
```

Step 4 – Investigate High Risks First

Now review the highest-risk events first by running:

```
index=ex10_hunt risk_score>=85 | table timestamp extracted_host
username event_type process_name detail risk_score | sort
timestamp
```

Write down the host and username that appear repeatedly in the high-risk results. In this dataset, the repeated pair is WKSTN-05 and kpatel.

Step 5 – Pivot

Pivot on that host and user together to isolate the suspicious sequence:

```
index=ex10_hunt extracted_host="WKSTN-05" username="kpatel" |
sort timestamp | table timestamp event_type process_name detail
risk_score
```

Group the suspicious events into stages. The schtasks.exe events suggest persistence setup, the reg.exe events suggest registry Run key persistence, and the after-hours powershell.exe network events suggest command-and-control or remote staging.

Step 6 – Documentation

Explain why this pattern is unusual on a normal user workstation. Your answer should mention repeated persistence changes and after-hours outbound traffic to a rare external host.

Write a short hunting conclusion that states whether you would escalate this system for containment and deeper review.

Module 11: Threat Intelligence Integration

Threat intelligence helps defenders recognize known adversary infrastructure, malware artifacts, and campaign patterns. At an entry level, students should understand that intelligence becomes useful when it is applied to detection, triage, and hunting rather than stored as a static list.

This module introduces practical threat intelligence concepts such as indicators of compromise, watchlists, retro-hunting, and alert enrichment. A list of IPs, domains, or hashes is valuable only when analysts compare it to real data and use the results to guide action.

Students should learn the strengths and limitations of IOC-based detection. It is helpful for finding known bad infrastructure, but it can age quickly and it does not replace behavioral analytics. Good defenders use threat intel to add context, prioritize investigations, and look for related activity across their environment.

This section also emphasizes communication. Threat intel is not only for analysts. A useful program can support management updates, defensive planning, and faster incident response by giving clear context on what a match likely means.

In practice, even a small IOC list can support several workflows: historical searching, watchlist matching, triage enrichment, and the creation of basic correlation rules.

By the end of this module, students should be able to explain how threat intelligence supports blue team operations and demonstrate at least one way to use indicator data inside a SIEM.

Exercise 11 – Threat Intel Driven Detection

Objective

Students use a small IOC list to build a basic watchlist workflow and plan retro-hunts in a SIEM.

Provided Dataset

- **Ex11_Threat_Intel_IOCs.csv**

Step 1 – Log Into Splunk

From the Windows 11 desktop, open a browser and browse to:

http://10.1.1.201

Username: Assigned by instructor
Password: BlueTeam2026!

Step 2 – Look at Table

Run the following search:

```
| inputlookup Ex11_Threat_Intel_IOCs.csv
```

The record count should be 10.

Step 3 – Separate Indicators By Type

Separate the indicators by type. You should see IP addresses, domains, URLs, hashes, and one email address.

Step 4 – Build a Query

Pick one IP indicator and write where you would search for it first. For example, 203.0.113.77 could appear in firewall, proxy, DNS, or endpoint network telemetry.

Pick one domain indicator and write where you would search for it first. For example, update-checkin.example could appear in DNS, web proxy, or email URL logs.

Write one example retro-hunt query for an IP indicator such as:

```
index=* "203.0.113.77"
```

Write one example retro-hunt query for a domain indicator such as:

```
index=* "update-checkin.example"
```

Step 5 – Tie It Together

First, run this query:

```
index=ex8_alerts source="Ex8_Real_World_Alert_Queue.csv" [ |
inputlookup Ex11_Threat_Intel_IOCs.csv | search ioc_type=ip |
rename indicator as dest_ip | fields dest_ip ] | table _time
alert_id src_ip dest_ip username alert_type description
```

Can you explain what this query does?

Now, run this query:

```
index=ex8_alerts source="Ex8_Real_World_Alert_Queue.csv" [ |
inputlookup Ex11_Threat_Intel_IOCs.csv | search ioc_type=ip |
rename indicator as src_ip | fields src_ip ] | table _time
alert_id src_ip dest_ip username alert_type description
```

Compare the results of these two queries to the data you collected during exercise 8.

Step 6 – Documentation

Explain which indicators would likely expire quickly. URLs, domains, and some IPs often change quickly, while file hashes can remain useful longer when the same sample is encountered again.

Finish with a short paragraph explaining how a SOC could use this IOC list for watchlists, triage enrichment, or retro-hunting.

Module 12: Network Traffic Analysis

Network traffic analysis helps defenders see communication patterns that may not be obvious from endpoint logs alone. Even when file names and process details are limited, the timing, direction, protocol, and size of traffic can reveal downloads, command-and-control activity, or data exfiltration.

This module introduces students to a practical analyst mindset: begin by establishing what looks normal, then look for repetition, unusual destinations, uncommon protocols, and patterns that suggest automation rather than human activity.

Students should pay particular attention to beaconing behavior. Repeated small outbound connections at regular intervals often indicate a compromised host checking in with an external controller. This kind of pattern is one of the clearest examples of how timing can be as important as content.

The module also reinforces the importance of baselining. A destination is not suspicious just because it appears often. Common software updates, cloud services, and internal infrastructure may be noisy but benign. Analysts must compare candidate traffic against the broader environment before drawing conclusions.

For students entering blue team roles, network traffic analysis is valuable because it strengthens their ability to validate suspicious behavior and explain why a communication pattern matters.

By the end of this module, students should be able to identify a likely C2 destination using repetition, timing, and context rather than relying only on signatures or reputation feeds.

Exercise 12 – Find the C2

Objective

Students analyze timestamped outbound traffic records to identify the most likely command-and-control destination based on repetition, timing, and byte patterns.

Provided Dataset

- **Ex12_Find_the_C2_Traffic_Fallback.csv**

Step 1 – Log Into Splunk

From the Windows 11 desktop, open a browser and browse to:

http://10.1.1.201

Username: Assigned by instructor
Password: BlueTeam2026!

Step 2 – Quick Look At Data

Run the following search:

```
index=ex12_traffic | stats count
```

The record count should be 60.

Step 3 – Count Destination IP Addresses

Start by counting how often each destination IP appears:

```
index=ex12_traffic | stats count by dest_ip | sort - count
```

Do not stop at the most frequent destination. Some common destinations in the dataset are likely normal services.

Step 4 – Filter Suspicious Destinations

Note: What makes a destination suspicious? Without a sense of what normal traffic looks like, it is almost impossible to determine what abnormal traffic is. A traffic baseline is critical for doing effective analysis.

Filter for the suspicious destination candidate 203.0.113.77 and review the traffic pattern:

```
index=ex12_traffic dest_ip="203.0.113.77" | table timestamp
src_ip dest_ip protocol bytes_out analysis_note | sort timestamp
```

Write down the source IP that is talking to 203.0.113.77. In this dataset, the source is 10.0.2.55.

Step 5 – Identify Why This Is Suspicious

Check the timestamps and note the pattern. The traffic occurs every 6 minutes and each row sends 540 bytes, which is more consistent with beaconing than normal browsing.

Compare that pattern with other destinations that have more varied timing and byte counts. Hint: replace the dest_ip field, or add a src_ip field to your query.

Step 6 – Documentation

Write your final answer naming 203.0.113.77 as the most likely C2 destination and explain that the regular interval and repeated small payload size are the key reasons.

Module 13: Network Defense Architecture

Blue team work is not only about detecting attacks after they begin. It also involves designing environments that are harder to compromise and easier to defend. Network defense architecture focuses on placing controls where they can prevent, detect, and contain attacker movement.

This module introduces defense in depth at a practical level. Students should think about segmentation, access control, administrative boundaries, logging placement, and containment options. The question is not whether one control can stop everything. The question is how multiple controls can work together to reduce risk.

A key design principle is limiting trust between zones. A DMZ system, user workstation, or externally accessible application should not have broad, unrestricted access to sensitive internal assets. When those trust boundaries are too open, a single compromise can quickly become a multi-system incident.

Students should also connect architecture decisions to blue team operations. Good design creates useful choke points for monitoring, clearer escalation paths, and fewer blind spots during an incident.

This section encourages students to recommend realistic improvements rather than idealized enterprise redesigns. A small team with limited budget can still improve resilience through segmentation, MFA, jump hosts, better logging, and prepared response actions.

By the end of this module, students should be able to explain how architectural controls can slow, block, and reveal attacker movement across a network.

Exercise 13 - DESIGN DEFENSE FOR AN ATTACK SCENARIO

Objective

Students design layered defenses to stop an attacker from moving from a DMZ-facing system into the internal network.

Scenario

You are part of a small blue team at a mid-sized company. The company has:

- employee workstations
- shared file servers
- Active Directory
- VPN access for remote users
- a small security team with limited time and budget

Management is worried that the company could be targeted by a real-world cyberattack. They want your team to think ahead and build a better defense before a major incident happens.

In this exercise, imagine an attacker uses a common attack path such as:

- tricking a user with phishing
- stealing or abusing credentials
- using tools like PowerShell or built-in Windows utilities
- moving from one system to another
- creating persistence to stay in the environment
- reaching sensitive files or business systems
- stealing data or launching ransomware

Your job is not to investigate logs from a completed attack.
Your job is to think like a defender and design how the company should protect itself.

You will explain:

- how the attacker might move through the environment
- which systems or users are most at risk
- what security controls should be added
- what logging and visibility the defenders need

- what detections should be created
- how the company should respond if the attack succeeds

Your recommendations should be practical and realistic. Assume the company does not have unlimited money, tools, or staff. Focus on the defenses that would make the biggest difference.

Step 1 – Review Scenario

Read the scenario and identify what the attacker can already do from the compromised DMZ system.

Step 2 – List Targets

List the first internal targets the attacker would probably try next, such as an application server, file share, domain controller, or backup server.

Step 3 – Draw Network Diagram

Draw a simple current-state network and mark where the DMZ currently connects to the internal network.

Step 4 – Add Preventive Control

Add at least one preventive control for each major attack path, such as firewall rules, jump servers, MFA for admin access, or network segmentation.

Step 5 – Add Detective Control

Add at least one detective control for each major path, such as admin logon alerts, east-west traffic monitoring, or EDR visibility on servers.

Step 6 – Add Response Control

Add at least one response control for each major path, such as host isolation, credential reset, or emergency firewall blocks.

Step 7 – Review Design

Review your design and make sure no single user workstation or DMZ host has direct unrestricted access to critical internal systems.

Step 8 – Document Your Work

Finish with a short paragraph explaining how your design would slow, block, or reveal lateral movement. For extra points, estimate a budget and timeline for implementation.

Capstone – Bringing It All Together

The capstone is designed to bring together the core skills of a junior blue team analyst into one realistic workflow. Students move from a single suspicious alert to a broader incident, using the same habits introduced throughout the course: validate the signal, pivot for context, build a timeline, assess scope, and recommend response actions.

Unlike earlier exercises that focus on one skill at a time, the capstone requires integration. Students must combine log analysis, SIEM searching, ATT&CK mapping, threat hunting, IOC identification, and defensive reasoning. This mirrors real blue team work, where incidents rarely arrive in neat categories.

The goal is not perfection. The goal is to demonstrate a repeatable investigation process and communicate findings clearly. A successful student should be able to explain what likely happened, what evidence supports that conclusion, what the immediate risks are, and what the organization should do next.

Students should also approach the capstone as a transition from classroom exercise to operational thinking. By the end of the course, they should feel more prepared to contribute in an entry-level SOC, incident response, or cyber defense role by asking good questions, documenting clearly, and supporting team decisions with evidence.

Use this final exercise as a synthesis of the full course: think like a defender, investigate methodically, map behavior to frameworks, and recommend practical steps that would help protect the environment in the future.

Capstone – Multi-Stage Incident Response

Objective

Students combine triage, host analysis, network analysis, and documentation to investigate a multi-stage intrusion that begins with phishing and ends with data exfiltration.

Provided Datasets

- **Capstone_Multi_Stage_IR_Dataset.csv**
- **Capstone_Multi_Stage_IR_Traffic_Fallback.csv**

Step 1 – Log Into Splunk

From the Windows 11 desktop, open a browser and browse to:

http://10.1.1.201

Username: Assigned by instructor
Password: BlueTEam2026!

Step 2 – Check Data

Run the following search:

```
index=capstone_ir | stats count

index=capstone_traffic | stats count
```

The record count should be 7 and 20 respectively.

Step 3 – Build Timeline

Start with the main event dataset and build a timeline with the query below:

```
index=capstone_ir | sort timestamp | table timestamp event_type
extracted_host username indicator detail severity
```

Step 4 – Explore Data

Read the first row and explain why the email event is suspicious.
The sender billing@secure-invoice.example should be treated as a phishing clue.

Step 5 – First Pivot

Pivot on username jrivera to follow the same user across the full incident:

```
index=capstone_ir username="jrivera" | sort timestamp | table
timestamp event_type extracted_host username indicator detail
severity
```

Step 6 – Identify Shift

Identify when the activity moves from the workstation to the server. In this dataset, the shift occurs when jrivera authenticates to SRV-FILE-02.

Step 7 – Second Pivot

Pivot on host SRV-FILE-02 to focus on the server stage:

```
index=capstone_ir extracted_host="SRV-FILE-02" | sort timestamp |
table timestamp event_type extracted_host username indicator
detail severity
```

Step 8 – Document Findings

Write down the indicators that suggest payload delivery, collection, and exfiltration. The most important external indicators are 203.0.113.88 for payload staging and 203.0.113.200 for external upload.

Step 9 – Verify Exfiltration

Now open the traffic dataset and verify the exfiltration destination with the query below:

```
index=capstone_traffic dest_ip="203.0.113.200" | table timestamp
src_ip dest_ip protocol bytes_out analysis_note | sort timestamp
```


Explain why repeated outbound traffic to 203.0.113.200 supports the exfiltration conclusion.

Step 10- Revise Timeline

Develop a short timeline with these stages: phishing, macro-launched PowerShell, outbound payload connection, server access, archive creation, external upload, and cloud sync tool execution.

Step 11 – Create Report

Create an incident report. Make sure to include what steps you took, evidence to support your conclusions, and recommend at least three immediate containment actions, such as isolating WKSTN-19 and SRV-FILE-02, disabling or resetting jrivera credentials, and blocking the external destinations involved in the attack.

